Segmentação e Medição Automática de Feridas

Roteiro de Apresentação e Explicação Técnica

Data de geração	6 de agosto de 2026
Domínio	Visão computacional aplicada à saúde — segmentação semântica e morfometria
Stack principal	Python · PyTorch · segmentation-models-pytorch · OpenCV · Albumentations
Base do documento	Análise dos 12 módulos Python, 3 arquivos de teste, README, requisitos, logs de execução e artefatos gerados
Dataset	Baixado e verificado: 2.760 pares imagem/máscara, 512×512 (ver seção 14)
Repositório Git	Não encontrado — o diretório não é um repositório Git

Sumário

1. Visão geral do projeto . . . 5

1.1 Objetivo . . . 5

1.2 Problema que o projeto resolve . . . 5

1.3 Contexto e origem . . . 5

1.4 Quem usaria o sistema . . . 6

1.5 Fluxo principal em uma frase . . . 6

2. Arquitetura . . . 7

2.1 Natureza do sistema . . . 7

2.2 Organização em camadas . . . 7

2.3 A decisão arquitetural mais relevante . . . 7

2.4 Fluxo dos dados no sistema . . . 8

3. Stack tecnológica . . . 10

3.1 Dependências de execução . . . 10

3.2 Dependências de desenvolvimento . . . 11

3.3 Linguagem e ambiente . . . 11

4. Funcionamento do sistema . . . 12

4.1 Entrada de dados . . . 12

4.2 Etapa de segmentação . . . 12

4.3 Treinamento . . . 12

4.4 Etapa de medição . . . 13

4.5 Conversão para unidades físicas . . . 14

4.6 Saídas produzidas . . . 15

5. Funcionalidades implementadas . . . 16

6. Minha implementação . . . 17

6.1 A evidência: dois blocos temporais . . . 17

6.2 O que já existia . . . 17

6.3 O que foi desenvolvido . . . 18

6.4 O que foi corrigido e melhorado no comportamento existente . . . 19

6.5 Como a compatibilidade foi preservada . . . 20

7. Detalhamento técnico . . . 21

7.1 Por que comprimento e largura vêm do retângulo de área mínima . . . 21

7.2 Por que as operações morfológicas estão desligadas . . . 21

7.3 O limiar duplo de área e a salvaguarda . . . 21

7.4 O contrato de “limpar uma vez só” . . . 21
7.5 Cálculo eficiente do diâmetro de Feret . . . 22
7.6 Reescala da máscara com interpolação por vizinho mais próximo . . . 22
7.7 Robustez das entradas . . . 22
7.8 Subamostragem nas curvas ROC e Precision-Recall . . . 22
7.9 O contrato efetivo da integração com a API do Kaggle . . . 22
7.10 Modelos de IA: entrada, processamento e resultado . . . 23
8. Desafios e soluções . . . 24
Desafio 1 — Uma imagem pode ter mais de uma ferida . . . 24
Desafio 2 — Limpar a máscara sem falsear a medida . . . 24
Desafio 3 — Medir comprimento e largura de forma justa . . . 24
Desafio 4 — Não inventar centímetros . . . 24
Desafio 5 — Evoluir a API sem quebrar o que já funcionava . . . 25
Desafio 6 — Testar sem GPU, sem dataset e sem modelo treinado . . . 25
Desafio 7 — Fazer a imagem anotada concordar com o CSV . . . 25
Desafio 8 — Rótulos legíveis em qualquer resolução e posição . . . 25
9. Roteiro de apresentação . . . 26
9.1 Abertura e contexto — 2 minutos . . . 26
9.2 Visão geral do pipeline — 2 minutos . . . 27
9.3 Arquitetura e organização — 2 minutos . . . 27
9.4 A segmentação — 2,5 minutos . . . 28
9.5 O núcleo do trabalho: a medição — 4 minutos . . . 29
9.6 A decisão sobre escala física — 2,5 minutos . . . 31
9.7 Qualidade e testes — 2,5 minutos . . . 32
9.8 Limitações — 1,5 minuto . . . 33
9.9 Encerramento — 1 minuto . . . 33
10. Roteiro de demonstração prática . . . 35
10.1 Plano A — demonstração ao vivo (exige preparação antecipada) . . . 35
10.2 Plano B — demonstração com o dataset real (recomendado) . . . 36
10.3 Comandos de referência . . . 37
11. Perguntas que podem ser feitas . . . 39
11.1 Arquitetura e decisões de projeto . . . 39
11.2 Modelo, treinamento e algoritmos . . . 39
11.3 Medição e algoritmos geométricos . . . 40
11.4 Dados, banco de dados e APIs . . . 41

11.5 Segurança . . . 41
11.6 Performance . . . 41
11.7 Qualidade, processo e problemas encontrados . . . 42
12. Melhorias futuras . . . 44
12.1 Prioridade alta — corrigem lacunas identificadas . . . 44
12.2 Prioridade média — ampliam a capacidade . . . 44
12.3 Prioridade estratégica — o que faria disso um produto . . . 44
13. Inconsistências e pontos de atenção . . . 46
13.1 Estado do ambiente . . . 46
13.2 Inconsistências no código . . . 46
13.3 Dependências e APIs depreciadas . . . 47
13.4 Documentação e artefatos . . . 47
13.5 Problemas descobertos ao rodar sobre o dataset real . . . 48
14. Execução real com o dataset Kaggle . . . 49
14.1 O que foi feito . . . 49
14.2 Composição do dataset . . . 49
14.3 Resultado da medição . . . 50
14.4 O argumento mais forte da sua apresentação . . . 50
14.5 Como reproduzir . . . 51
15. Portabilidade: rodar em qualquer computador . . . 52
15.1 A falha grave: o projeto estava quebrado em instalações novas . . . 52
15.2 Versões travadas nos dois sentidos . . . 52
15.3 Baixar os dados sem a pilha de deep learning . . . 52
15.4 Compatibilidade de AMP entre gerações do PyTorch . . . 52
15.5 Credenciais do Kaggle: três mecanismos . . . 53
15.6 verificar_ambiente.py — o diagnóstico . . . 53
15.7 Resumo das mudanças . . . 53

1. Visão geral do projeto

> **Como ler este documento**
>
> Todo o conteúdo a seguir foi extraído da leitura direta dos arquivos do projeto. Onde uma informação não pôde ser confirmada nos arquivos, isso está dito explicitamente com a marcação **"não encontrado"** ou **"não identificado"**. As seções 6 e 13 tratam separadamente do que é atribuível ao seu trabalho e do que são pontos de atenção do código.

1.1 Objetivo

O projeto é um sistema completo, em Python e PyTorch, que recebe a fotografia de uma ferida e devolve duas coisas: a **delimitação automática** da região lesionada (segmentação) e um **conjunto de medidas morfológicas objetivas** dessa região — área, comprimento, largura, perímetro, circularidade, solidez, orientação e outras. O resultado sai em três formatos: um relatório CSV com uma linha por ferida, uma imagem anotada com as marcações e os valores sobrepostos, e um bloco de texto legível para console ou log.

O sistema não para na segmentação. A segmentação é meio; o produto final é a **medição**. Essa distinção é importante para a apresentação, porque é onde está concentrado o trabalho mais recente e onde estão as decisões técnicas mais interessantes.

1.2 Problema que o projeto resolve

A avaliação de feridas crônicas — úlceras por pressão, pé diabético, úlceras venosas — depende do acompanhamento das dimensões da lesão ao longo do tempo, porque a variação de área é o principal indicador de que um tratamento está funcionando. Na prática clínica corrente, essa medição é feita manualmente, com régua descartável ou decalque em papel acetato. Isso traz três problemas concretos:

- **Subjetividade:** a borda da ferida é definida a olho por quem mede, e dois profissionais chegam a valores diferentes para a mesma lesão.
- **Baixa reprodutibilidade:** a mesma pessoa, medindo em dias diferentes, não reproduz exatamente o mesmo contorno.
- **Medida grosseira:** a régua fornece comprimento e largura aproximados; área real, perímetro e formato (circularidade, solidez) não são obtidos manualmente com precisão.

O projeto ataca exatamente isso: a borda passa a ser definida por um modelo treinado, de forma determinística e repetível, e as medidas passam a ser calculadas por geometria computacional sobre essa borda — não estimadas visualmente.

1.3 Contexto e origem

O projeto é construído sobre o dataset público `leoscode/wound-segmentation-images`, hospedado no Kaggle, cujo download é automatizado via API oficial do Kaggle no módulo `dataset.py`. **Esse dataset foi baixado e verificado** durante a elaboração deste documento: 664,8 MB, 2.760 pares imagem/máscara em resolução uniforme de 512×512, provenientes de três fontes (`fusc`: 1.210, `wsnet`: 1.176, `medetec`: 374). Os números reais estão na **seção 14**.

A leitura dos docstrings revela que o projeto foi desenvolvido em **etapas numeradas**, e essa numeração está registrada no próprio código. É um bom mapa mental para a apresentação:

Etapa	Onde está no código	O que faz
1	`dataset.py`	Download automático via API do Kaggle e organização das pastas
2	`explore_data.py` / `dataset.py`	Exploração dos dados, estatísticas e histogramas

Etapa	Onde está no código	O que faz
3	`dataset.py`	Transformações e augmentations (Albumentations)
4	`models.py`	Fábrica de modelos de segmentação
5	`losses.py`	Função de perda Dice + BCE
6	`metrics.py`	Métricas de segmentação
7	`train.py`	Loop de treinamento completo
8	`predict.py`	Predição em uma imagem
9, 10 e 11	`measure.py`	Medição morfológica, escala física e relatório
12 e 13	`evaluate.py`	Avaliação no conjunto de teste e curvas
14	—	**Não encontrada** nos arquivos: nenhum módulo declara a etapa 14
15	`batch_inference.py`	Inferência em lote sobre uma pasta

1.4 Quem usaria o sistema

Pelo tipo de saída que o sistema produz (CSV por ferida, imagem anotada, medidas morfológicas), os perfis de uso são:

- **Equipes de enfermagem e estomaterapia** — acompanhamento objetivo da evolução de feridas crônicas entre curativos.
- **Pesquisa clínica** — extração padronizada de medidas de coortes de imagens, com processamento em lote via `batch_inference.py`.
- **Telemedicina / triagem remota** — a foto é o único insumo necessário.

> **Ressalva que você deve fazer na apresentação**
>
> O sistema é um trabalho técnico e não um dispositivo médico validado. Não há no projeto nenhuma validação clínica, aprovação regulatória ou estudo de concordância com medição manual. Diga isso espontaneamente: mostra maturidade e evita que a banca levante o ponto antes de você.

1.5 Fluxo principal em uma frase

Uma imagem RGB entra; é redimensionada para 512×512 e normalizada; a U-Net produz um mapa de probabilidade; o limiar de 0,5 gera uma máscara binária; a máscara volta à resolução original; o pós-processamento remove ruído e preenche buracos; cada região conectada vira uma ferida; cada ferida é medida por geometria computacional; e, se — e somente se — um fator de escala for informado, as medidas em pixels são convertidas para centímetros.

2. Arquitetura

2.1 Natureza do sistema

Antes de descrever a estrutura, é importante estabelecer o que este sistema **não é**, porque isso costuma ser a primeira pergunta de quem avalia:

Componente	Situação no projeto
Front-end	**Não encontrado.** Não há interface web, desktop ou mobile. A interação é por linha de comando.
Back-end / API HTTP	**Não encontrado.** Não há Flask, FastAPI, Django ou qualquer servidor. Os módulos se comunicam por chamada direta de função Python.
Banco de dados	**Não encontrado.** Não há SQL, ORM ou NoSQL. A persistência é o sistema de arquivos: PNG, CSV e checkpoints `.pt`.
Docker / infraestrutura	**Não encontrado.** Não há Dockerfile, docker-compose, manifesto de CI/CD ou script de deploy.
Integrações externas	Uma: a **API do Kaggle**, usada apenas para baixar o dataset. Detalhada na seção 7.9.

O sistema é, portanto, uma **aplicação de pipeline em lote, orientada a scripts** — o formato natural para um projeto de visão computacional focado em treinar um modelo e extrair medidas. Apresente isso como uma decisão, não como uma lacuna: a ausência de camada web mantém o projeto reprodutível e executável no Google Colab sem adaptação.

2.2 Organização em camadas

Os 12 módulos Python da raiz se organizam em cinco camadas com responsabilidades bem separadas. Nenhum módulo acumula duas responsabilidades:

Camada	Módulos	Responsabilidade
Configuração	`config.py`	Ponto único de verdade: caminhos, seed, hiperparâmetros e parâmetros de medição, todos em *dataclasses*. Nenhum número mágico espalhado pelo código.
Dados	`dataset.py` `explore_data.py`	Download, organização, pareamento imagem/máscara, divisão determinística, augmentations e o `Dataset` do PyTorch.

Modelo	`models.py`
	`losses.py`
	`metrics.py`
		Fábrica de arquiteturas, função de perda combinada e métricas de segmentação.

Execução	`train.py`
	`predict.py`
	`batch_inference.py`
	`evaluate.py`
		Os quatro pontos de entrada do sistema: treinar, prever em uma imagem, prever em lote e avaliar no conjunto de teste.

Domínio	`measure.py`	O núcleo de valor do projeto: transforma uma máscara binária em medidas morfológicas, converte unidades e produz relatório e anotação visual.
Suporte	`utils.py`	Seeds, logging, seleção de dispositivo e primitivas de visualização (overlay e contorno).

2.3 A decisão arquitetural mais relevante

Há uma decisão de acoplamento que vale destacar na apresentação, porque não é óbvia e tem consequência prática direta: **o módulo de medição não depende de PyTorch**.

`measure.py` importa apenas `cv2`, `numpy`, `pandas` e `utils`. E `utils.py` foi escrito de modo que `torch` seja importado **dentro** das funções que realmente precisam dele — `seed_everything()` e `get_device()` — e não no topo do arquivo. O próprio docstring de `utils.py` declara essa intenção.

A consequência é concreta: a suíte de testes de medição roda com apenas `opencv`, `numpy` e `pandas` instalados, sem baixar PyTorch, sem GPU e sem checkpoint treinado. Isso torna o teste do núcleo do projeto rápido e viável em qualquer máquina. É um bom exemplo de decisão técnica para citar quando perguntarem sobre testabilidade.

2.4 Fluxo dos dados no sistema

Existem dois fluxos distintos. O **fluxo de treinamento**, executado uma vez, e o **fluxo de inferência**, executado a cada nova imagem.

Fluxo de treinamento

#	Etapa	Módulo	Saída produzida
1	Download e organização	`dataset.py`	`dataset/images/` e `dataset/masks/`
2	Pareamento e divisão 70/15/15	`dataset.py`	Objeto `DatasetSplits` determinístico
3	Augmentation e normalização	`dataset.py`	Tensores 3×512×512 normalizados
4	Treino com AMP, early stopping	`train.py`	`checkpoints/best_<run>.pt` + histórico CSV + TensorBoard
5	Avaliação no teste	`evaluate.py`	Matriz de confusão, ROC, PR e relatórios CSV

Fluxo de inferência (o que você vai demonstrar)

#	Transformação	Estado do dado	Responsável
1	Leitura e conversão BGR→RGB	Imagem RGB, resolução original	`predict.predict_mask`
2	Resize 512×512 + normalização ImageNet	Tensor 1×3×512×512	`dataset.get_eval_transforms`
3	Passagem pela U-Net	Logits 1×1×512×512	`models.build_model`
4	Sigmoide + limiar 0,5	Máscara binária 512×512	`predict.predict_mask`
5	Reescala com INTER_NEAREST	**Máscara na resolução original**	`predict.predict_mask`
6	Limpeza: buracos e componentes pequenos	Máscara limpa	`measure.clean_mask`
7	Rotulagem de componentes conectados	N regiões independentes	`measure.measure_wounds`
8	Medição geométrica por região	N objetos `WoundMeasurements`	`measure._measure_component`
9	Conversão de unidades (opcional)	Campos `*_cm` preenchidos	`measure.PixelScale`
10	Relatório e anotação	CSV, PNG anotado, texto	`measure` + `predict`

O passo 5 merece destaque na apresentação: a máscara volta à **resolução original** antes de qualquer medição. Isso significa que todas as medidas em pixels se referem à imagem que o usuário forneceu, e não à imagem

redimensionada internamente pela rede. Sem isso, um fator de escala informado pelo usuário estaria errado por um fator arbitrário.

3. Stack tecnológica

Sobre as versões desta tabela

As versões abaixo são as **restrições declaradas** em `requirements.txt` e `requirements-dev.txt` — todas no formato "maior ou igual a". O projeto **não possui lockfile** (não há `poetry.lock`, `Pipfile.lock` nem `requirements.lock`), então a versão resolvida não era reproduzível a partir do repositório.

Isso foi corrigido (seção 15). O `requirements.txt` passou a usar faixas com **limite superior**, e o novo `requirements-verificado.txt` fixa a combinação exata de versões com a qual os **54 testes passam**. A pilha completa foi instalada e validada nesta máquina: torch 2.13.0+cpu, smp 0.5.0, torchmetrics 1.9.0, albumentations 2.0.8, OpenCV 5.0.0, NumPy 2.5.1, pandas 3.0.5, scikit-learn 1.9.0.

3.1 Dependências de execução

Tecnologia	Versão declarada	Onde é usada	Finalidade no projeto
PyTorch	>= 2.1.0	`train.py`, `predict.py`, `models.py`	Framework de deep learning: tensores, autograd, treino, AMP e inferência.
segmentation-models-pytorch (smp)	>= 0.3.3	`models.py`	Fornece as arquiteturas de segmentação já montadas (U-Net, U-Net++, DeepLabV3+, FPN) com encoders pré-treinados. Evita reimplementar a rede do zero.
Albumentations	>= 1.4.0	`dataset.py`	Pipeline de *data augmentation*. Aplica a mesma transformação geométrica à imagem e à máscara simultaneamente — requisito essencial em segmentação.
OpenCV (opencv-python-headless)	>= 4.9.0	`measure.py`, `utils.py`, `dataset.py`	**Motor de toda a medição:** componentes conectados, contornos, `minAreaRect`, fecho convexo, momentos, além de leitura/escrita de imagens e desenho das anotações.
NumPy	>= 1.26.0	Todos os módulos	Representação das máscaras como arrays e cálculos vetorizados (Feret, filtros de área).
pandas	>= 2.1.0	`measure.py`, `evaluate.py`, `train.py`	Montagem dos relatórios tabulares e escrita dos CSVs.
Matplotlib	>= 3.8.0	`predict.py`, `evaluate.py`, `explore_data.py`	Grids de comparação visual, histogramas, curvas ROC/PR e matriz de confusão.
scikit-learn	>= 1.4.0	`evaluate.py`	Matriz de confusão, curva ROC e curva Precision-Recall.
TorchMetrics	>= 1.3.0	`metrics.py`	Implementações numericamente robustas de IoU, Precision, Recall, F1 e Accuracy, com acumulação correta entre lotes.
TensorBoard	>= 2.15.0	`train.py`	Registro de métricas e taxa de aprendizado por época para acompanhamento visual.
tqdm	>= 4.66.0	`train.py`, `evaluate.py`, `batch_inference.py`	Barras de progresso nos laços de treino e inferência.

Tecnologia	Versão declarada	Onde é usada	Finalidade no projeto
kaggle	>= 1.6.0	`dataset.py`	Cliente oficial da API do Kaggle para o download automático do dataset.
torchvision	>= 0.16.0	**Nenhum import direto**	Declarada em `requirements.txt`, mas **nenhum módulo do projeto a importa**. Provável dependência transitiva de `smp`; mantida.
scikit-image	—	**Removida**	Estava declarada e **não era usada em nenhum arquivo**. Foi retirada do `requirements.txt` para aliviar a instalação (seção 15).

3.2 Dependências de desenvolvimento

Tecnologia	Versão declarada	Onde é usada	Finalidade
pytest	>= 8.0.0	`tests/`	Executa os 54 testes automatizados. Declarado em `requirements-dev.txt`, que reaproveita `requirements.txt` via `-r` — separando corretamente o que é necessário para rodar do que é necessário para testar.

3.3 Linguagem e ambiente

Item	Evidência encontrada
Versão do Python (documentada)	O `README.md` instrui criar o ambiente com `python3.11`.
Versão do Python (efetiva)	Os arquivos compilados em `__pycache__/` têm o sufixo `cpython-314`, indicando que o projeto foi **executado em Python 3.14**. Há divergência entre o documentado e o utilizado — ver seção 13.
Sintaxe moderna	Todos os módulos usam `from __future__ import annotations` e sintaxe `X | None`, compatível com Python 3.10+.
Suporte a GPU	`utils.get_device()` detecta CUDA automaticamente e faz *fallback* silencioso para CPU. O treino ativa *mixed precision* apenas quando há GPU.
Google Colab	O `README.md` declara compatibilidade sem alterações de código. Nenhum notebook (`.ipynb`) foi encontrado no projeto.

4. Funcionamento do sistema

Esta seção detalha o pipeline completo em três blocos: como a rede segmenta, como a máscara vira número e como o número vira centímetro.

4.1 Entrada de dados

O sistema aceita uma imagem RGB em qualquer resolução e em qualquer um dos formatos listados em `dataset.IMAGE_EXTENSIONS`: `.png`, `.jpg`, `.jpeg`, `.bmp`, `.tif` e `.tiff`. Nenhum metadado é exigido — e, importante, nenhum metadado é **lido**: o sistema não tenta extrair escala de EXIF.

No treinamento, a entrada é um par (imagem, máscara). O pareamento é feito em `dataset.list_image_mask_pairs()` pelo nome-base do arquivo, com remoção dos sufixos `_mask` e `-mask`. A máscara é binarizada com o limiar fixo de 127.

4.2 Etapa de segmentação

Pré-processamento

A imagem é redimensionada para 512×512 (`DATA_CFG.image_size`) e normalizada com a média e o desvio-padrão do ImageNet — `[0.485, 0.456, 0.406]` e `[0.229, 0.224, 0.225]`. A normalização usa as estatísticas do ImageNet, e não do dataset de feridas, porque o encoder é pré-treinado no ImageNet e espera receber entradas na mesma distribuição em que foi treinado.

O modelo

A configuração padrão é uma **U-Net com encoder ResNet34 pré-treinado no ImageNet**, montada por `models.build_model()`. A escolha da U-Net para segmentação médica é convencional e bem fundamentada: as conexões de salto (*skip connections*) entre o caminho de contração e o de expansão preservam a informação espacial fina necessária para delimitar a borda com precisão — exatamente o que este projeto precisa, já que é a borda que determina a área e o perímetro medidos.

O uso de *transfer learning* (encoder pré-treinado) é a decisão que torna o treinamento viável: datasets médicos são pequenos, e treinar um encoder do zero com poucas centenas de imagens levaria a sobreajuste. O encoder já sabe extrair bordas e texturas; o treino ajusta essa capacidade ao domínio das feridas.

A fábrica em `models.py` é intencionalmente parametrizável e **valida a entrada**: aceita quatro arquiteturas (`unet`, `unetplusplus`, `deeplabv3plus`, `fpn`) e três encoders (`resnet34`, `efficientnet-b0`, `efficientnet-b3`), levantando `ValueError` com a lista de opções válidas se receber algo fora disso. Trocar de arquitetura é editar uma linha em `config.py` — não há mudança de código.

Saída da rede

A rede emite **um único canal de logits** (`classes=1`, `activation=None`). A sigmoide não é aplicada dentro do modelo: ela é aplicada na função de perda (via `BCEWithLogitsLoss`) e na inferência. Isso é deliberado e é um bom ponto técnico para mencionar — aplicar a sigmoide dentro da BCE usa o truque do *log-sum-exp*, que é numericamente estável, enquanto calcular sigmoide e depois log separadamente pode gerar overflow.

O limiar de 0,5 sobre a probabilidade produz a máscara binária de uma classe: ferida contra fundo. **Não há detecção de objetos, bounding boxes nativas nem classificação de tipo de tecido** — é segmentação binária pura.

4.3 Treinamento

Hiperparâmetro	Valor	Justificativa
Função de perda	0,5·Dice + 0,5·BCE	A BCE é estável e otimiza pixel a pixel, mas é dominada pelo fundo em imagens desbalanceadas. **No dataset real, a ferida ocupa em mediana apenas 1,55% da imagem** (média 3,85%) — ou seja, prever "tudo fundo" já daria BCE baixa. A Dice otimiza diretamente a sobreposição da região e é insensível a esse desbalanceamento. Combinadas, dão estabilidade e foco na região de interesse.
Otimizador	Adam, lr 1e-4	Taxa conservadora, adequada a *fine-tuning* de encoder pré-treinado.
Weight decay	1e-5	Regularização leve.
Épocas	60 (máximo)	Limite superior; o *early stopping* normalmente encerra antes.
Early stopping	paciência 10	Interrompe após 10 épocas sem melhora de `val_dice`.
Scheduler	ReduceLROnPlateau fator 0,5 · paciência 5	Reduz a taxa pela metade quando o Dice de validação estagna.
Mixed precision (AMP)	ativa em GPU	Reduz memória e acelera o treino; desativada automaticamente em CPU.
Gradient clipping	norma 1,0	Evita passos instáveis, especialmente sob AMP.
Batch size	8	Compatível com 512×512 em GPUs de memória média.
Divisão dos dados	70 / 15 / 15	Treino, validação e teste.
Seed	42	Fixa Python, NumPy e PyTorch, e torna a divisão sempre idêntica.

O critério de seleção do melhor modelo é o **Dice de validação**, e não a perda. Isso é correto para segmentação: a perda combinada mistura dois objetivos e não é diretamente interpretável, enquanto o Dice mede exatamente o que interessa — a sobreposição entre a máscara prevista e a real. O checkpoint salvo em `train.py` guarda os pesos, a configuração do modelo, a época e o `val_dice`, o que permite a `predict.load_model_from_checkpoint()` reconstruir a arquitetura correta sem que o usuário precise informá-la.

4.4 Etapa de medição

Esta é a parte central do projeto e está inteiramente em `measure.py`. O princípio que rege todo o módulo está declarado no seu docstring e deve ser dito na apresentação: **a medição parte exclusivamente da máscara produzida pela segmentação; nada é estimado visualmente a partir da foto.**

Passo 1 — Limpeza da máscara (`clean_mask`)

1. **Abertura morfológica** — *desativada por padrão*.
2. **Fechamento morfológico** — *desativado por padrão*.
3. **Preenchimento de buracos** — regiões internas classificadas como fundo (tecido de granulação, fibrina) são reincorporadas à ferida. Implementado em `fill_mask_holes()`, que redesenha cada região a partir do seu contorno externo. Altera apenas o interior: a borda externa fica intacta.
4. **Remoção de componentes pequenos** — descarta regiões com área menor que `max(50 px, 0,02% da imagem)`. Com uma salvaguarda: se *todos* os componentes ficarem abaixo do limiar, o maior é preservado, para não descartar uma ferida legítima em imagem de baixa resolução.

> **A decisão técnica mais defensável do módulo**
>
> As operações morfológicas estão **desligadas por padrão** — e o código explica por quê, em comentário no próprio `config.py`: elas **deslocam a borda** e, portanto, enviesam justamente a área e o perímetro que se quer medir. Uma abertura elíptica 3×3 remove os cantos de um retângulo. Como o produto do projeto é a medida, e não a aparência da máscara, a limpeza é feita pelo filtro de área — que é exato e preserva a borda. As operações continuam disponíveis via configuração para máscaras serrilhadas. **Este é o melhor ponto isolado para defender na apresentação.**

Passo 2 — Separação das feridas

Cada componente conectado sobrevivente (conectividade 8) é tratado como uma **ferida independente** e recebe um `wound_id` sequencial, atribuído em ordem decrescente de área. O parâmetro `max_wounds` (padrão 10) limita quantas feridas são reportadas; o total detectado permanece registrado no campo `num_connected_components` e o truncamento é gravado em log.

Passo 3 — Medição de cada ferida

São calculadas **17 métricas por ferida**, em `_measure_component()`:

Métrica	Campo	Como é obtida
Área	`area_pixels`	Contagem exata de pixels do componente
Perímetro	`perimeter_pixels`	`cv2.arcLength` sobre o contorno externo completo
Comprimento	`length_pixels`	**Maior lado** do retângulo de área mínima
Largura	`width_pixels`	**Menor lado** do mesmo retângulo (perpendicular)
Bounding box	`bounding_box`	`(x, y, w, h)` alinhada aos eixos
Centroide	`centroid`	Momentos de imagem: `m10/m00`, `m01/m00`
Diâmetro de Feret máx.	`feret_max_pixels`	Maior distância entre dois pontos do fecho convexo
Diâmetro equivalente	`equivalent_diameter_pixels`	`sqrt(4·área/π)` — círculo de mesma área
Razão comprimento/largura	`length_width_ratio`	`length / width`
Razão de aspecto	`aspect_ratio`	`w / h` da bounding box
Circularidade	`circularity`	$4\pi \cdot \text{área} / \text{perímetro}^2$ (1 = círculo perfeito)
Solidez	`solidity`	Área ÷ área do fecho convexo (mede concavidade)
Extensão	`extent`	Área ÷ área da bounding box
Orientação	`orientation_deg`	Ângulo do eixo maior, em [0, 180)
Área convexa	`convex_area_pixels`	`cv2.contourArea` do fecho convexo
Percentual da imagem	`percent_image_area`	`100 × área / (H·W)`
Nº de componentes	`num_connected_components`	Total de regiões detectadas na imagem

4.5 Conversão para unidades físicas

Este é o ponto de honestidade técnica do projeto, e é o que mais impressiona quando bem explicado. **Por padrão, todas as medidas saem em pixels e px².**

A razão é factual: o dataset utilizado não contém informação de escala — não há EXIF confiável, nem marcador de referência nas fotos, nem registro da distância câmera-ferida. Sem isso, converter pixels em centímetros exigiria **inventar** um fator. O projeto se recusa a fazer isso: `config.DEFAULT_PIXELS_POR_CM` é `None` e todos os campos `*_cm` permanecem `None` até que o usuário forneça a escala explicitamente.

A classe `PixelScale` centraliza a conversão e oferece quatro construtores, todos convergindo para a mesma unidade canônica interna (pixels por centímetro):

Construtor	Entrada	Caso de uso
`from_pixels_per_cm`	Fator direto	Escala já conhecida e calibrada
`from_pixels_per_mm`	Fator em mm	Conveniência para quem trabalha em milímetros
`from_reference_object`	Tamanho em px + tamanho real em cm	**Método recomendado em campo:** régua, adesivo ou etiqueta na foto
`from_dpi`	Resolução em DPI	Imagens de escâner ou digitalização planar

Na linha de comando, tanto `predict.py` quanto `batch_inference.py` expõem `--pixels-por-cm`, `--pixels-por-mm` e o par `--referencia-pixels` / `--referencia-cm`. A função `scale_from_options()` resolve a precedência entre eles e devolve `None` quando nada é informado.

4.6 Saídas produzidas

Artefato	Gerado por	Conteúdo
`<nome>_medicoes.csv`	`predict.py`	Uma linha por ferida, com todas as métricas
`<nome>_medicoes.png`	`predict.py`	Imagem anotada: contorno, bounding box, retângulo mínimo, eixos de comprimento e largura, centroide e rótulo com os valores
`<nome>_overlay.png`	`predict.py`	Máscara translúcida sobre a foto
`<nome>_comparacao.png`	`predict.py`	Grid: original | máscara real (se houver) | prevista | overlay | medições
`medicoes_lote.csv`	`batch_inference.py`	Uma linha por imagem (ferida principal)
`medicoes_por_ferida.csv`	`batch_inference.py`	Uma linha por ferida detectada
`relatorio_por_imagem.csv`	`evaluate.py`	Medidas + Dice por imagem do conjunto de teste
`relatorio_por_ferida.csv`	`evaluate.py`	Medidas + Dice por ferida
Figuras de avaliação	`evaluate.py`	Matriz de confusão, curva ROC, curva Precision-Recall

5. Funcionalidades implementadas

Todas as funcionalidades listadas abaixo foram confirmadas no código. Nenhuma é planejada ou parcial.

#	Funcionalidade	Onde	Como funciona
1	Download automático do dataset	`dataset.py`	Autentica na API do Kaggle, baixa, descompacta os `.zip` e organiza imagens e máscaras em pastas separadas. É idempotente: se os dados já existem, pula o download.
2	Organização automática de pastas	`dataset.py`	Varre recursivamente o download bruto e separa imagens de máscaras por palavras-chave (`mask`, `label`, `gt`, `annotation`) no caminho ou no nome do arquivo.
3	Exploração de dados	`explore_data.py`	Contagens, resolução média, histogramas de área absoluta e percentual, e grid de exemplos aleatórios com máscara sobreposta.
4	Divisão determinística	`dataset.py`	70/15/15 embaralhado com seed fixa — a mesma imagem cai sempre na mesma partição, garantindo que o conjunto de teste nunca contamine o treino entre execuções.
5	Augmentation para segmentação	`dataset.py`	Flips horizontal e vertical, rotação até 35°, ShiftScaleRotate, CLAHE, brilho/contraste, desfoque gaussiano e variação de matiz/saturação. A máscara sofre as mesmas transformações geométricas que a imagem.
6	Quatro arquiteturas trocáveis	`models.py`	U-Net, U-Net++, DeepLabV3+ e FPN, com três encoders. Troca por configuração, com validação explícita da opção informada.
7	Treinamento completo	`train.py`	AMP, early stopping, ReduceLROnPlateau, gradient clipping, checkpoint do melhor modelo, TensorBoard e histórico em CSV.
8	Avaliação com curvas	`evaluate.py`	IoU, Precision, Recall, F1, Accuracy, matriz de confusão pixel a pixel, curva ROC e curva Precision-Recall, mais relatórios por imagem e por ferida.
9	Predição em imagem única	`predict.py`	Segmenta, mede, anota e salva quatro artefatos, imprimindo as medidas em texto.
10	**Medição multi-ferida**	`measure.py`	Cada componente conectado vira uma ferida independente, com 17 métricas próprias, ordenadas por área.
11	**Conversão de unidades opcional**	`measure.py`	Quatro construtores de escala; sem escala informada, o sistema reporta pixels e declara isso explicitamente, em vez de inventar um fator.
12	**Anotação visual das medidas**	`measure.py`	Desenha contorno, bounding box, retângulo mínimo, eixos de comprimento e largura, centroide e um rótulo com os valores — com o rótulo reposicionado automaticamente para não sair da imagem.
13	Inferência em lote	`batch_inference.py`	Processa uma pasta inteira, com tratamento de erro por imagem: uma foto corrompida é registrada em log e o lote continua.
14	**Suíte de testes automatizados**	`tests/`	54 testes cobrindo medição, escala, robustez, relatórios, visualização e integração ponta a ponta.

Os itens em negrito concentram o trabalho mais recente — ver seção 6.

6. Minha implementação

> **Nota metodológica — leia antes de usar esta seção**
>
> O projeto **não é um repositório Git**: não existe pasta `.git`, portanto **não há histórico de commits, autoria ou diffs** para consultar. A atribuição abaixo foi reconstruída a partir de três evidências objetivas: (a) as **datas de modificação** dos arquivos, que se separam em dois blocos nítidos; (b) os **docstrings do próprio código**, que marcam explicitamente funções como "legado", "histórico" e "API antiga" em oposição às novas; e (c) a **existência de testes** escritos especificamente para o comportamento novo. Isso é forte, mas é inferência: nada aqui prova autoria. Onde a atribuição é menos certa, o texto diz **"possível contribuição"**. Apresente desta forma — se perguntarem, diga com naturalidade que o projeto não está versionado em Git e que essa é uma melhoria que você reconhece como necessária.

6.1 A evidência: dois blocos temporais

As datas de modificação dos arquivos separam o projeto em dois momentos claramente distintos, com três semanas entre eles:

Arquivo	Última modificação	Interpretação
`dataset.py`	14/07, 21:04	**Base pré-existente** — intocado
`explore_data.py`	14/07, 21:04	**Base pré-existente** — intocado
`models.py`	14/07, 21:04	**Base pré-existente** — intocado
`losses.py`	14/07, 21:04	**Base pré-existente** — intocado
`metrics.py`	14/07, 21:05	**Base pré-existente** — intocado
`train.py`	14/07, 21:05	**Base pré-existente** — intocado
`requirements.txt`	14/07, 21:07	**Base pré-existente** — intocado
`utils.py`	06/08, 11:06	Modificado
`evaluate.py`	06/08, 11:08	Modificado
`tests/conftest.py`	06/08, 11:09	**Novo**
`config.py`	06/08, 11:11	Modificado
`predict.py`	06/08, 11:16	Modificado
`batch_inference.py`	06/08, 11:16	Modificado
`tests/test_measure.py`	06/08, 11:16	**Novo**
`tests/test_pipeline_integration.py`	06/08, 11:18	**Novo**
`requirements-dev.txt`	06/08, 11:18	**Novo**
`README.md`	06/08, 11:20	Reescrito e expandido
`measure.py`	06/08, 11:20	**Reescrito e muito expandido**

O padrão é inequívoco: **toda a pilha de treinamento e modelagem permanece como estava; todo o bloco de medição, escala, apresentação e testes é trabalho do segundo momento.**

6.2 O que já existia

A base pré-existente é um projeto de segmentação competente e completo: download e organização do dataset, augmentations, fábrica de modelos com quatro arquiteturas, perda Dice+BCE, métricas com TorchMetrics, e um loop de treinamento maduro com AMP, early stopping, scheduler, checkpointing e TensorBoard.

Já existia também uma **medição rudimentar**: a função `measure_wound()`, que media apenas o **maior** componente conectado da máscara crua, sem pós-processamento, e levantava `ValueError` se a máscara estivesse vazia. Ela produzia 10 campos — área, perímetro, bounding box, centroide, circularidade, razão de aspecto, área convexa, solidez, percentual da imagem e contagem de componentes — mais dois campos opcionais em centímetros.

A evidência de que esses eram os campos originais está no próprio docstring da dataclass `WoundMeasurements`, que declara: *"os campos até* `perimeter_cm` *são os originais do projeto e mantêm exatamente a mesma semântica"*.

6.3 O que foi desenvolvido

A. Medição de múltiplas feridas por imagem

Antes: uma imagem com três feridas produzia uma única medida — a da maior. As demais eram silenciosamente ignoradas.

Depois: a nova função `measure_wounds()` rotula todos os componentes conectados, mede cada um individualmente, atribui um `wound_id` sequencial em ordem decrescente de área e devolve uma lista. Quando não há ferida, devolve **lista vazia** em vez de lançar exceção — mudança de contrato deliberada, porque "nenhuma ferida detectada" é um resultado clínico legítimo, não um erro de programa.

B. Sete novas métricas morfológicas

Foram acrescentadas: comprimento e largura invariantes à rotação, diâmetro de Feret máximo, diâmetro equivalente, razão comprimento/largura, extensão e orientação. Todas com valor padrão na dataclass, para que código antigo que construísse `WoundMeasurements` continuasse funcionando — comportamento verificado pelo teste `test_wound_measurements_legacy_construction`.

C. Sistema de escala física (`PixelScale`)

Classe imutável (`frozen=True`) com quatro construtores alternativos, validação no `__post_init__` que rejeita valores não-positivos, infinitos e NaN, e um método `resolve()` que normaliza os três tipos de entrada aceitos pelas funções públicas — um objeto `PixelScale`, um número simples ou `None`. É esse `resolve()` que permitiu adicionar o sistema de escala sem quebrar a assinatura antiga `pixels_por_cm=38.0`.

D. Pós-processamento configurável (`clean_mask`)

Preenchimento de buracos, filtro de área com limiar duplo (absoluto e relativo à imagem), salvaguarda contra descartar a ferida inteira, e operações morfológicas disponíveis mas **desligadas por padrão** com justificativa escrita. Acompanha a nova dataclass `MeasurementConfig` em `config.py`.

E. Anotação visual das medidas

`annotate_measurements()` desenha, por ferida, o contorno, a bounding box, o retângulo de área mínima, os eixos de comprimento e largura, o centroide e um bloco de rótulo com os valores. A espessura das linhas e o tamanho da fonte **escalam com a resolução da imagem**, e o rótulo é reposicionado automaticamente quando não cabe acima da ferida ou escaparia da borda.

F. Camada de relatório e apresentação

`wounds_to_rows()` converte medições em linhas de CSV — e, quando não há ferida, emite uma linha marcada com `wound_id=0` e a observação "nenhuma ferida detectada", para que **a imagem não desapareça do relatório**. `measurements_to_row()` omite os campos usados apenas para desenho, mantendo o CSV legível. `format_measurements_text()` produz saída em texto que mostra centímetros só quando há escala e, caso

contrário, avisa explicitamente que os valores estão em pixels.

G. Integração nos três pontos de entrada

- `predict.py` — quatro novas opções de linha de comando para escala, painel de medições no grid de comparação, geração do PNG anotado e do CSV por ferida.
- `batch_inference.py` — segundo CSV consolidado por ferida, imagens anotadas por item do lote e as mesmas opções de escala.
- `evaluate.py` — `collect_predictions()` passou a devolver também os registros por ferida, gerando o `relatorio_por_ferida.csv`.

H. Suíte de testes (54 testes) — o item de maior peso

Criada do zero. Nenhum teste existia antes: a pasta `tests/` inteira é do segundo bloco temporal.

Arquivo	Testes	Cobertura
`tests/conftest.py`	7 fixtures	Máscaras sintéticas de **geometria conhecida analiticamente**: vazia, retângulo 120×60, círculo r=70, duas feridas, formato em "L", máscara ruidosa com buraco interno e retângulo rotacionado 30°.
`tests/test_measure.py`	45	Dimensões exatas, círculo, Feret, múltiplas feridas, limite `max_wounds`, forma irregular, **invariância à rotação**, remoção de ruído, preenchimento de buracos, idempotência da limpeza, conversão de unidades, rejeição de escalas inválidas, máscara vazia, formatos 0/1 e 0/255, booleana e float, baixa resolução, relatórios, texto e anotação visual.
`tests/test_pipeline_integration.py`	9	Fluxo ponta a ponta segmentação → medição usando um **segmentador determinístico** (`_StubSegmenter`) que dispensa checkpoint treinado. Valida também que a base pré-existente não quebrou.

O `_StubSegmenter` é o detalhe mais elegante da suíte e vale ser citado: é um `nn.Module` que devolve logits altos dentro de um retângulo conhecido em coordenadas relativas e baixos fora dele. Com ele, o teste conhece a resposta correta analiticamente e pode verificar o pipeline inteiro **sem treinar nada, sem GPU e sem checkpoint**.

I. Refatoração de `utils.py` para testabilidade

O import de `torch` foi movido do topo do arquivo para dentro das duas funções que precisam dele. Foi essa mudança que permitiu testar a medição sem instalar a pilha de deep learning. Acrescentou-se também `draw_contour()`, usada pela anotação.

J. Documentação

O `README.md` atual tem 363 linhas e documenta o funcionamento da segmentação, a tabela completa das 17 métricas com a fórmula de cada uma, a justificativa do `minAreaRect`, o sistema de escala, a recomendação prática de fotografia com marcador de referência, as APIs, e — o mais importante — uma seção explícita de **limitações conhecidas** com oito itens.

6.4 O que foi corrigido e melhorado no comportamento existente

Aspecto	Antes	Depois
Feridas múltiplas	Só a maior era medida; as demais, ignoradas	Todas são medidas e identificadas por `wound_id`
Ausência de ferida	`ValueError` — interrompia o processamento	Lista vazia e linha marcada no relatório; o lote continua

Aspecto	Antes	Depois
Ruído de segmentação	Medido como se fosse ferida	Filtrado por área, com salvaguarda para não descartar a ferida real
Buracos internos	Reduziam a área e inflavam o perímetro	Preenchidos por `fill_mask_holes()`
Comprimento e largura	Não existiam como métrica	Do retângulo de área mínima, invariantes à rotação
Escala física	Parâmetro numérico solto, sem validação	Classe `PixelScale` validada, com quatro construtores
Saída visual	Overlay simples da máscara	Imagem anotada com geometria e valores medidos
Verificação	**Nenhum teste**	**54 testes automatizados**

6.5 Como a compatibilidade foi preservada

Este é o ponto que demonstra cuidado de engenharia, e você deve destacá-lo. Toda a expansão foi feita **sem quebrar nada do que já existia**, por quatro mecanismos:

1. A função antiga `measure_wound()` foi **mantida com comportamento idêntico** — continua medindo só o maior componente, continua sem aplicar limpeza e continua levantando `ValueError` em máscara vazia. Dois testes fixam esse contrato: `test_legacy_measure_wound_ignores_cleanup` e `test_empty_mask_legacy_raises`.
2. Os novos campos da dataclass têm **valores padrão**, de modo que qualquer construção antiga continua válida.
3. O parâmetro `pixels_por_cm` manteve o nome e passou a aceitar também um objeto `PixelScale`, graças ao `resolve()`.
4. Os artefatos antigos continuam sendo gerados: `batch_inference.py` ainda escreve `_mask.png`, `_overlay.png` e `medicoes_lote.csv` — o teste `test_batch_inference_generates_both_reports` verifica explicitamente que os arquivos antigos ainda existem.

7. Detalhamento técnico

Esta seção explica as decisões técnicas que sustentam a implementação. É o material para responder “por que você fez assim?”.

7.1 Por que comprimento e largura vêm do retângulo de área mínima

Problema. Como medir “comprimento” e “largura” de uma forma irregular?

Solução ingênua e por que falha. Usar a largura e a altura da bounding box. Mas a bounding box é alinhada aos eixos da imagem: uma ferida alongada e inclinada a 45° produz uma caixa muito maior que a ferida, e a medida passa a depender de como a foto foi enquadrada — girar a câmera mudaria o resultado.

Solução adotada. `cv2.minAreaRect()`, em `_rotated_dimensions()`. O retângulo de área mínima é **invariante à rotação**: o maior lado é o comprimento e o menor lado é a largura perpendicular a ele. Isso reproduz exatamente a convenção clínica de “maior comprimento × maior largura perpendicular”.

Como isso é verificado. O teste `test_rotation_invariance` constrói um retângulo de 140×50 rotacionado 30° e verifica que o comprimento medido é 140 e a largura é 50, dentro de tolerância — enquanto assegura que a bounding box alinhada aos eixos é comprovadamente maior. O teste documenta a decisão.

A função também deriva os **eixos** do retângulo por trigonometria a partir do centro e do ângulo, o que permite desenhar as linhas de comprimento e largura sobre a imagem — é o que torna a anotação visual verificável a olho nu.

7.2 Por que as operações morfológicas estão desligadas

Problema. Máscaras de segmentação vêm com ruído: pixels isolados, pequenos falsos positivos, bordas irregulares. O reflexo automático é aplicar abertura e fechamento morfológicos.

Por que isso seria errado aqui. Operações morfológicas **deslocam a borda**. Uma abertura com elemento elíptico 3×3 arredonda cantos e erode contornos; um fechamento dilata. Em um sistema cujo produto é a **medida da área e do perímetro**, isso introduz viés sistemático justamente na grandeza de interesse — e o viés não é aleatório, é sempre na mesma direção.

Solução adotada. O ruído é removido por **filtro de área sobre componentes conectados**, que é exato: ou o componente é descartado inteiro, ou é preservado inteiro, sem tocar em um único pixel da borda da ferida real. As operações morfológicas permanecem implementadas e disponíveis via `MeasurementConfig`, para quem tiver máscaras serrilhadas — a decisão é um padrão, não uma amputação.

7.3 O limiar duplo de área e a salvaguarda

O limiar de descarte é `max(min_component_area_px, min_component_area_ratio × área da imagem)`, ou seja, `max(50 px, 0,02%)` por padrão. A combinação de um piso absoluto com um piso relativo faz o filtro se adaptar à resolução: em uma imagem de 4000×3000, 50 px é ruído irrelevante, e o critério relativo assume.

A salvaguarda é o detalhe que evita um erro grave: **se todos os componentes ficarem abaixo do limiar, o maior é preservado**. Sem isso, uma ferida pequena em imagem de baixa resolução seria completamente descartada e o sistema reportaria “nenhuma ferida” — um falso negativo silencioso. O teste `test_tiny_mask_is_not_discarded_entirely` fixa esse comportamento com uma ferida de 16 px, abaixo do limiar de 50.

7.4 O contrato de “limpar uma vez só”

Há uma sutileza de integração que vale explicar, porque é o tipo de detalhe que diferencia código pensado de código montado.

Em `predict.py` e `batch_inference.py`, a máscara é limpa **uma vez** com `clean_mask()`, e essa mesma máscara limpa é usada para duas coisas: alimentar a medição (com `cleanup=False`, para não limpar de novo) e desenhar a anotação. O resultado é que **o contorno desenhado na imagem corresponde exatamente à máscara de onde saíram os números**. Se a limpeza fosse aplicada duas vezes, ou se a anotação usasse a máscara crua, a imagem e o CSV poderiam discordar — e o usuário não teria como saber qual está certo.

Esse contrato é verificado por dois testes: `test_cleaned_mask_measures_identically`, que confirma que limpar-e-medir dá o mesmo resultado que medir-com-limpeza, e `test_clean_mask_is_idempotent`, que garante que aplicar a limpeza duas vezes não altera mais a máscara.

7.5 Cálculo eficiente do diâmetro de Feret

O diâmetro de Feret máximo é a maior distância entre dois pontos quaisquer da borda. Calculado ingenuamente sobre todos os pontos do contorno, o custo é $O(n^2)$, e um contorno extraído com `CHAIN_APPROX_NONE` pode ter milhares de pontos.

A implementação em `_feret_max()` calcula sobre o **fecho convexo**. A justificativa geométrica é sólida: o par de pontos mais distantes de um conjunto pertence necessariamente ao seu fecho convexo. O custo cai para $O(h^2)$, com h igual ao número de vértices do fecho — tipicamente uma ou duas ordens de grandeza menor, sem nenhuma perda de exatidão.

7.6 Reescala da máscara com interpolação por vizinho mais próximo

Em `predict.predict_mask()`, a máscara de 512×512 volta à resolução original com `cv2.INTER_NEAREST`. A escolha é obrigatória, não estética: qualquer interpolação suavizadora (bilinear, bicúbica) criaria valores intermediários entre 0 e 1 em uma máscara que precisa ser estritamente binária, exigindo um segundo limiar arbitrário e alterando a área medida. `INTER_NEAREST` preserva a natureza binária.

7.7 Robustez das entradas

A função interna `_binarize()` normaliza qualquer formato de máscara — 0/1, 0/255, `float`, `bool` — para `uint8` 0/1 com uma única expressão. Isso elimina uma classe inteira de bugs: máscara vinda do modelo (0/1), máscara lida de PNG (0/255) e máscara construída em teste (booleana) percorrem o mesmo caminho. Dois testes cobrem isso: `test_mask_with_255_values_is_accepted` e `test_boolean_and_float_masks_are_accepted`.

7.8 Subamostragem nas curvas ROC e Precision-Recall

Em `evaluate.collect_predictions()`, os vetores de probabilidade e rótulo são subamostrados com o passo `[::37]` antes de alimentar as curvas. A razão é de memória: o conjunto de teste inteiro, a 512×512 por imagem, produz dezenas de milhões de pixels, e `roc_curve` ordena todo o vetor. O passo 37, por ser primo, evita alinhamento com a largura da imagem, o que impediria amostrar sempre a mesma coluna. **Importante:** a subamostragem afeta apenas as curvas; as métricas agregadas (IoU, F1, Dice) são calculadas por TorchMetrics sobre **todos** os pixels.

7.9 O contrato efetivo da integração com a API do Kaggle

Existe uma única integração externa no projeto, e a pergunta “como ela funciona?” tem resposta apenas no código de integração. O contrato efetivamente utilizado é:

Aspecto	Contrato utilizado pelo projeto
Cliente	`from kaggle.api.kaggle_api_extended import KaggleApi` — importado **dentro** da função, com `try/except ImportError` que converte a falha em mensagem instrutiva
Autenticação	`api.authenticate()` — lê `~/.kaggle/kaggle.json` ou as variáveis `KAGGLE_USERNAME` e `KAGGLE_KEY`

Aspecto	Contrato utilizado pelo projeto
Chamada	`api.dataset_download_files(slug, path=..., unzip=False, quiet=False)`
Recurso	`leoscode/wound-segmentation-images`
Pós-processamento	O projeto descompacta manualmente (`unzip=False`) todos os `*.zip` e apaga os arquivos compactados
Idempotência	Se `dataset/raw/` já contém arquivos, o download é pulado — salvo se `force=True`
Credenciais	**Nunca ficam no repositório.** Vêm do ambiente ou do arquivo padrão do Kaggle

7.10 Modelos de IA: entrada, processamento e resultado

Resumo do pipeline de IA em uma tabela, útil para responder rapidamente na apresentação:

Fase	Formato do dado	Operação
Entrada	Imagem RGB, H×W×3, `uint8`	Resize 512×512, normalização ImageNet, conversão para tensor
Processamento	Tensor 1×3×512×512, `float32`	U-Net + encoder ResNet34: encoder extrai características em múltiplas escalas, decoder reconstrói a resolução usando as conexões de salto
Saída bruta	Logits 1×1×512×512	Sigmoide → probabilidade por pixel → limiar 0,5
Pós-processamento	Máscara binária H×W	Reescala INTER_NEAREST, preenchimento de buracos, filtro de área
Resultado	N × 17 métricas + artefatos	Componentes conectados, geometria por região, conversão opcional de unidades, CSV e PNG anotado

8. Desafios e soluções

Sete desafios técnicos reais, cada um com o problema, a causa, a solução adotada e o resultado verificável. Esta é a seção mais valiosa para a apresentação — bancas avaliam capacidade de resolver problemas, não de listar funcionalidades.

Desafio 1 — Uma imagem pode ter mais de uma ferida

- **Problema:** a medição existente reportava apenas o maior componente; feridas satélite desapareciam do relatório sem qualquer aviso.
- **Causa:** `measure_wound()` chamava `_largest_component_mask()` e descartava o restante da máscara.
- **Solução:** `measure_wounds()` rotula todos os componentes com `connectedComponentsWithStats`, mede cada um isoladamente e ordena por área decrescente, atribuindo `wound_id` sequencial. O limite `max_wounds` protege contra máscaras patológicas, e o total real fica registrado em `num_connected_components`.
- **Resultado:** verificado por `test_multiple_wounds_are_measured_individually` e `test_multiple_wounds_have_distinct_centroids`. Em execução real registrada em log, uma imagem produziu 9 componentes distintos, todos medidos.

Desafio 2 — Limpar a máscara sem falsear a medida

- **Problema:** a máscara precisa de limpeza, mas toda limpeza altera a geometria — e a geometria *é* o resultado do sistema.
- **Causa:** operações morfológicas deslocam a borda; o viés incide diretamente sobre área e perímetro.
- **Solução:** separar os dois tipos de defeito. **Buracos internos** são preenchidos por reconstrução a partir do contorno externo — altera só o interior. **Ruído externo** é removido por filtro de área — descarta o componente inteiro, sem tocar na borda da ferida. Morfologia fica disponível, mas desligada por padrão.
- **Resultado:** `test_noise_is_removed_before_measuring` confirma que a bounding box da ferida real permanece **exatamente** `(100, 80, 120, 60)` após a limpeza, e `test_holes_are_filled` confirma que a área volta ao valor exato de 7200 px².

Desafio 3 — Medir comprimento e largura de forma justa

- **Problema:** a bounding box alinhada aos eixos superestima grosseiramente feridas inclinadas, tornando a medida dependente do enquadramento da foto.
- **Causa:** a bounding box é, por definição, alinhada aos eixos da imagem.
- **Solução:** `cv2.minAreaRect()`, invariante à rotação, com o maior lado como comprimento e o menor como largura perpendicular.
- **Resultado:** `test_rotation_invariance` comprova que um retângulo 140×50 girado 30° é medido como 140×50, e não pelas dimensões infladas da caixa alinhada aos eixos.

Desafio 4 — Não inventar centímetros

- **Problema:** o resultado clinicamente útil é em centímetros, mas o dataset não contém nenhuma informação de escala.
- **Causa:** as fotos não têm EXIF confiável, marcador de referência nem registro da distância câmera-ferida. Qualquer conversão automática seria um número inventado — e um número inventado em contexto clínico é pior que nenhum número.
- **Solução:** o sistema **nunca infere escala**. `DEFAULT_PIXELS_POR_CM` é `None`, os campos `*_cm` permanecem `None`, e a saída em texto declara explicitamente que os valores estão em pixels. Quando o usuário fornece a escala, `PixelScale` a valida e converte, oferecendo quatro formas de informá-la — sendo a recomendada o objeto de referência fotografado no mesmo plano da ferida.

- **Resultado:** `test_without_scale_physical_fields_are_none` verifica campo a campo que nenhum valor físico é fabricado, e `test_pixel_scale_rejects_invalid_values` garante que escalas inválidas (zero, negativa, NaN, infinito) falham imediatamente em vez de gerar números sem sentido.

Desafio 5 — Evoluir a API sem quebrar o que já funcionava

- **Problema:** `measure_wound()` já era chamada por `evaluate.py` e `batch_inference.py`. Mudar seu comportamento quebraria a avaliação e invalidaria comparações com resultados anteriores.
- **Causa:** a nova semântica desejada (múltiplas feridas, limpeza automática, lista vazia em vez de exceção) é incompatível com a antiga.
- **Solução: duas APIs convivendo por escolha explícita.** A antiga foi congelada; a nova nasceu ao lado. Novos campos ganharam valores padrão; o parâmetro de escala passou a aceitar mais tipos sem mudar de nome, via `PixelScale.resolve()`.
- **Resultado:** `test_legacy_measure_wound_ignores_cleanup` prova que a API antiga continua sem limpar (área de 7200–36 px, com o buraco preservado) e que continua vendo o ruído. Os artefatos antigos do lote seguem sendo gerados, conforme `test_batch_inference_generates_both_reports`.

Desafio 6 — Testar sem GPU, sem dataset e sem modelo treinado

- **Problema:** validar o pipeline exigiria um checkpoint treinado, o dataset baixado e a pilha de deep learning instalada — inviável para uma suíte de testes rápida.
- **Causa:** acoplamento natural entre medição e inferência.
- **Solução:** três medidas combinadas. (a) O import de `torch` em `utils.py` foi adiado para dentro das funções, permitindo importar `measure.py` com apenas `opencv` e `numpy`. (b) As máscaras de teste são **sintéticas, com geometria conhecida analiticamente** — sabe-se de antemão que o retângulo tem 7200 px². (c) O `_StubSegmenter` substitui o modelo treinado por um `nn.Module` determinístico, e os testes de integração são pulados automaticamente com `pytest.importorskip` quando a pilha de deep learning não está presente.
- **Resultado:** 45 testes de medição rodam sem PyTorch; 9 testes de integração validam o fluxo completo sem checkpoint. Os logs do projeto registram **quatro execuções completas e bem-sucedidas** da suíte em 6 de agosto.

Desafio 7 — Fazer a imagem anotada concordar com o CSV

- **Problema:** se a anotação visual fosse desenhada sobre uma máscara diferente da que foi medida, a imagem e o relatório discordariam — e o usuário não teria como saber qual acreditar.
- **Causa:** a limpeza acontece dentro de `measure_wounds()` por padrão, mas a função de desenho recebe a máscara separadamente.
- **Solução:** o parâmetro `cleanup`. Os scripts limpam a máscara **uma vez**, passam-na para a medição com `cleanup=False` e usam a **mesma** máscara na anotação.
- **Resultado:** `test_cleaned_mask_measures_identically` verifica que os dois caminhos produzem área, bounding box e perímetro idênticos, e `test_clean_mask_is_idempotent` garante a estabilidade da operação.

Desafio 8 — Rótulos legíveis em qualquer resolução e posição

- **Problema:** texto desenhado com tamanho fixo fica ilegível em imagens grandes e cobre a imagem inteira em imagens pequenas; e uma ferida colada na borda superior teria seu rótulo desenhado fora da imagem.
- **Causa:** `cv2.putText` trabalha em coordenadas absolutas de pixel.
- **Solução:** em `annotate_measurements()`, a espessura das linhas e o `font_scale` são derivados da menor dimensão da imagem dividida por 512. Em `_draw_label_block()`, o tamanho do bloco é calculado com `cv2.getTextSize` e a posição é ajustada: se não couber acima da ferida, o bloco vai para baixo, e a posição final é limitada aos limites da imagem com `np.clip`. Um retângulo escurecido é aplicado atrás do texto para garantir contraste sobre qualquer fundo.

- **Resultado:** `test_annotate_wound_at_image_border` exercita uma ferida colada no canto superior esquerdo e confirma que a imagem resultante mantém as dimensões corretas, sem estouro.

9. Roteiro de apresentação

Roteiro completo para uma apresentação de **18 a 20 minutos**, dividido em nove blocos. Os textos em destaque são para falar em voz alta — não precisam ser decorados palavra por palavra, mas seguem a ordem lógica que sustenta o argumento.

> **Regra de ouro para esta apresentação**
> O projeto tem duas metades: uma pilha de segmentação bem construída e um sistema de medição. Se você gastar o tempo todo falando de U-Net, vai apresentar o que **não** é o seu diferencial. Reserve **metade do tempo** para a medição, a escala e os testes — é ali que estão as decisões técnicas defensáveis e o trabalho recente.

9.1 Abertura e contexto — 2 minutos

O que mostrar

- Nada na tela ainda, ou apenas o `README.md` aberto no topo.
- Se possível, uma das imagens anotadas de `outputs/predictions/` já aberta em outra aba, para revelar ao final deste bloco.

> **O QUE FALAR**
>
> *Eu vou apresentar um sistema de segmentação e medição automática de feridas. Antes do código, deixa eu explicar o problema, porque ele define todas as decisões técnicas que eu tomei.*
>
> *No acompanhamento de feridas crônicas — úlcera por pressão, pé diabético, úlcera venosa — o que indica se o tratamento está funcionando é a variação do tamanho da lesão ao longo do tempo. Hoje, na prática, isso é medido com régua descartável ou decalque em papel acetato. Isso traz três problemas: a borda da ferida é definida a olho, então dois profissionais chegam a valores diferentes; a mesma pessoa não reproduz o mesmo contorno em dias diferentes; e a régua só dá comprimento e largura aproximados — área real, perímetro e formato não saem daí.*
>
> *O que este sistema faz é substituir esses dois passos. A borda passa a ser definida por um modelo de segmentação, de forma determinística e repetível. E as medidas passam a ser calculadas por geometria computacional sobre essa borda. O resultado é um CSV com uma linha por ferida e uma imagem anotada com os valores desenhados em cima.*
>
> *Uma coisa que eu quero deixar clara desde o começo: isto é um trabalho técnico, não é um dispositivo médico validado. Não há aqui validação clínica nem estudo de concordância com medição manual.*

Pontos técnicos importantes

- O produto final é a **medida**, não a máscara. A segmentação é meio.
- Antecipar a ressalva sobre validação clínica desarma a pergunta antes de ela vir.

Possíveis perguntas neste ponto

Por que não usar simplesmente uma régua na foto e medir manualmente no computador?

Porque isso resolve a escala, mas não resolve a subjetividade da borda — que é a principal fonte de variação. O sistema resolve a borda automaticamente; a régua na foto continua sendo o método recomendado para a escala, e é justamente por isso que existe o construtor `PixelScale.from_reference_object()`.

Isso substitui o profissional de saúde?

Não. Ele padroniza e acelera a medição, e produz um registro objetivo e auditável. A decisão clínica continua sendo humana, e a imagem anotada existe justamente para que o profissional confira visualmente se o contorno detectado faz sentido.

9.2 Visão geral do pipeline — 2 minutos

O que mostrar

- A seção “Fluxo completo” no topo do `README.md` — há um diagrama em texto que resume o pipeline em cinco setas.
- Em seguida, a estrutura de pastas, também no `README.md`.

> **O QUE FALAR**
>
> *O fluxo é linear e tem cinco estágios. A imagem entra; a rede de segmentação produz uma máscara binária separando ferida de fundo; essa máscara passa por um pós-processamento; cada região conectada que sobrevive é tratada como uma ferida independente; e cada ferida é medida individualmente. No final, se — e só se — um fator de escala for informado, as medidas em pixels são convertidas para centímetros.*
>
> *Esse “se e só se” não é um detalhe, é uma decisão de projeto central, e eu volto nela daqui a pouco.*
>
> *A estrutura reflete esse fluxo: cada etapa é um módulo com uma responsabilidade só. Nenhum arquivo faz duas coisas.*

Pontos técnicos importantes

- Citar que o projeto foi construído em **etapas numeradas**, e que essa numeração está registrada nos docstrings — mostra organização.
- Plantar aqui a expectativa sobre a escala; você vai colher no bloco 9.6.

9.3 Arquitetura e organização — 2 minutos

O que mostrar

- `config.py` — rolar pelas quatro *dataclasses*.
- A lista de arquivos da raiz do projeto.

> **O QUE FALAR**
>
> *A arquitetura é de pipeline em lote, orientada a scripts. Não há front-end, não há API HTTP e não há banco de dados — a persistência é o sistema de arquivos: PNG, CSV e checkpoints. Isso é intencional para um projeto de visão computacional: mantém tudo reprodutível e executável no Colab sem adaptação nenhuma.*
>
> *A configuração é centralizada em `config.py`, em quatro dataclasses: dados, modelo, treinamento e medição. A ideia é não ter número mágico espalhado pelo código — para trocar de arquitetura, mudar o tamanho da imagem ou ajustar o filtro de ruído da medição, eu edito um arquivo só.*
>
> *Tem uma decisão de acoplamento aqui que eu acho que vale destacar. O módulo de medição não depende de PyTorch. Ele importa só OpenCV, NumPy e pandas. E o `utils.py` foi escrito de forma que o import do torch aconteça dentro das funções que realmente precisam dele, e não no topo do arquivo. A consequência prática é que eu consigo testar todo o núcleo de medição sem instalar a pilha de deep learning, sem GPU e sem modelo treinado. Isso vai ficar importante quando eu chegar nos testes.*

Pontos técnicos importantes

- Nomear as quatro dataclasses: `DataConfig`, `ModelConfig`, `TrainConfig`, `MeasurementConfig`.

- O import adiado de `torch` é uma decisão de **testabilidade**, e o docstring do arquivo declara isso.

Possíveis perguntas neste ponto

Por que não expor isso como uma API REST?

Não era o objetivo do trabalho, que é o pipeline de visão computacional. Mas a arquitetura já está preparada para isso: `predict_single_image()` é uma função pura que recebe caminhos e devolve um dicionário. Envolver isso num endpoint FastAPI seriam poucas linhas, e a mudança ficaria contida em um arquivo novo, sem tocar em nada do que existe.

Por que não usar um banco de dados em vez de CSV?

Para o volume e o padrão de uso atuais — processamento em lote e análise posterior em pandas ou Excel — o CSV é suficiente e não adiciona dependência de infraestrutura. Um banco faria sentido se o sistema passasse a acompanhar pacientes ao longo do tempo, que é justamente uma das melhorias que eu listo no final.

9.4 A segmentação — 2,5 minutos

O que mostrar

- `models.py` — a fábrica e o dicionário de arquiteturas.
- `losses.py` — a perda combinada.
- `train.py` — a função `train_model()`, rolando rapidamente.

> **O QUE FALAR**
>
> *A segmentação usa uma U-Net com encoder ResNet34 pré-treinado no ImageNet, montada com a biblioteca segmentation-models-pytorch.*
>
> *A U-Net é a escolha convencional para segmentação médica, e tem um motivo concreto: as conexões de salto entre o caminho de contração e o de expansão preservam a informação espacial fina, que é o que permite delimitar a borda com precisão. E, nesse projeto, a borda é literalmente o resultado — é ela que determina a área e o perímetro que eu vou medir depois.*
>
> *O encoder é pré-treinado porque datasets médicos são pequenos. Treinar do zero com poucas centenas de imagens levaria a sobreajuste. O encoder já sabe extrair bordas e texturas; o treino só ajusta isso ao domínio das feridas.*
>
> *A perda é uma combinação de Dice com entropia cruzada binária, meio a meio. A entropia cruzada sozinha é dominada pelo fundo, porque a ferida ocupa uma fração pequena da imagem — nos dados desse projeto, algo em torno de poucos por cento. A Dice otimiza diretamente a sobreposição da região. Juntas, dão estabilidade numérica e foco na região que interessa.*
>
> *O treino tem mixed precision, early stopping, redução de taxa de aprendizado em platô, gradient clipping e checkpoint do melhor modelo. E o critério de seleção do melhor modelo é o Dice de validação, não a perda — porque a perda mistura dois objetivos e não é diretamente interpretável, enquanto o Dice mede exatamente o que eu quero maximizar.*

Pontos técnicos importantes

- A rede emite **logits**, sem sigmoide — que é aplicada na perda, via `BCEWithLogitsLoss`, por estabilidade numérica (log-sum-exp).
- Quatro arquiteturas e três encoders trocáveis por configuração, com validação explícita.
- O checkpoint guarda os **pesos e a configuração do modelo**, o que permite reconstruir a arquitetura certa na inferência sem o usuário informar nada.

Possíveis perguntas neste ponto

Por que U-Net e não DeepLabV3+ ou uma arquitetura mais recente?

A U-Net é o padrão consolidado em segmentação médica por causa das conexões de salto, e é a configuração padrão do projeto. Mas a escolha não está fixa no código: a fábrica em `models.py` suporta U-Net++, DeepLabV3+ e FPN, e trocar é mudar uma string em `config.py`. Uma comparação sistemática entre elas está na minha lista de trabalhos futuros — eu não tenho esse benchmark rodado, então não vou afirmar qual é melhor aqui.

Por que 0,5 e 0,5 nos pesos da perda?

É o ponto de partida neutro, e está parametrizado em `TrainConfig` justamente para ser ajustado. Não fiz busca de hiperparâmetro sobre esses pesos — se tivesse feito, eu teria o resultado para mostrar.

Por que 512×512?

É um compromisso entre preservar detalhe de borda e caber na memória com batch 8. E, importante: a máscara volta para a resolução original antes de qualquer medição, então esse redimensionamento não contamina as medidas.

9.5 O núcleo do trabalho: a medição — 4 minutos

O que mostrar

- Uma imagem anotada de `outputs/predictions/` — **abra em tela cheia**. É o momento visual mais forte da apresentação.
- `measure.py` — as funções `clean_mask()`, `measure_wounds()` e `_rotated_dimensions()`.
- Um dos CSVs de medição, aberto ao lado da imagem.

O QUE FALAR

Essa é a parte central do trabalho, e está toda no `measure.py`*.*

O princípio que rege o módulo inteiro é: a medição parte exclusivamente da máscara produzida pela segmentação. Nada é estimado visualmente a partir da foto.

Antes de medir, a máscara passa por uma limpeza. E aqui está a decisão técnica de que eu mais gosto no projeto. O reflexo natural, quando se tem uma máscara ruidosa, é aplicar abertura e fechamento morfológico. Eu deixei as duas ***desligadas por padrão****. O motivo é que operações morfológicas deslocam a borda — uma abertura elíptica 3 por 3 remove os cantos de um retângulo. Num sistema em que o produto final é justamente a área e o perímetro, isso introduz um viés sistemático exatamente na grandeza que eu quero medir.*

Então eu separei os dois tipos de defeito. Buraco interno — tecido de granulação que o modelo classificou como fundo dentro da ferida — é preenchido reconstruindo a região a partir do contorno externo: isso muda só o interior, a borda fica intacta. Ruído externo é removido por filtro de área sobre componentes conectados: ou o componente inteiro é descartado, ou é preservado inteiro. Nenhum pixel da borda da ferida real é tocado. As operações morfológicas continuam implementadas e disponíveis por configuração, para quem tiver máscaras serrilhadas — o que eu fiz foi escolher um padrão, não remover a opção.

Depois da limpeza, cada componente conectado vira uma ferida independente, com um identificador próprio, ordenado por área. Antes, o sistema media só o maior componente e as feridas satélite sumiam do relatório sem aviso.

De cada ferida eu extraio dezessete métricas. Vou destacar uma, porque ela ilustra o tipo de decisão que o projeto exigiu: comprimento e largura. O caminho ingênuo seria usar a largura e a altura da bounding box. Mas a bounding box é alinhada aos eixos da imagem — uma ferida alongada inclinada a quarenta e cinco graus geraria uma caixa muito maior que a ferida, e a medida passaria a depender de como a foto foi enquadrada. Eu uso o retângulo de área mínima, que é invariante à rotação: o maior lado é o comprimento, o menor lado é a largura perpendicular. Isso reproduz exatamente a convenção clínica de maior comprimento por maior largura perpendicular.

E aqui na imagem dá para verificar isso a olho nu: o retângulo em ciano é o retângulo de área mínima, e a linha vermelha e a azul são os eixos de comprimento e largura desenhados sobre a ferida. O amarelo é a bounding box alinhada aos eixos — dá para ver a diferença entre os dois.

Pontos técnicos importantes

- As 17 métricas: área exata por contagem de pixels, perímetro por `arcLength`, Feret pelo fecho convexo, circularidade, solidez, extensão e orientação.
- O limiar duplo de área: `max(50 px, 0,02% da imagem)` — adapta-se à resolução.
- A salvaguarda: se todos os componentes ficarem abaixo do limiar, o maior é preservado, para não produzir falso negativo silencioso.
- O Feret é calculado sobre o fecho convexo: $O(h^2)$ em vez de $O(n^2)$, sem perda de exatidão.

Possíveis perguntas neste ponto

Como você garante que o filtro de área não está descartando uma ferida real e pequena?

Por duas coisas. O limiar é duplo — absoluto e relativo à área da imagem — então ele se adapta à resolução. E existe uma salvaguarda explícita: se *todos* os componentes ficarem abaixo do limiar, o maior é preservado. Sem isso, uma ferida pequena em imagem de baixa resolução viraria um “nenhuma ferida detectada”, que é um falso negativo silencioso — o pior tipo de erro nesse contexto. Tem um teste específico para isso, com uma ferida de 16 pixels, abaixo do limiar de 50.

E se duas feridas estiverem encostadas uma na outra?

Elas são fundidas em um único componente conectado e medidas como uma só. Isso está documentado como limitação conhecida no README. Separá-las exigiria uma abordagem de instâncias — *watershed* ou um modelo de segmentação de instâncias — e está na lista de melhorias futuras.

A circularidade não deveria dar exatamente 1 para um círculo?

Deveria, no contínuo. Na prática dá um pouco menos, porque o `arcLength` percorre a borda em escada, pixel a pixel, e superestima levemente o perímetro. Como a circularidade tem o perímetro ao quadrado no denominador, ela fica correspondentemente subestimada. Isso está registrado nas limitações conhecidas, e o teste do círculo aceita a faixa de 0,85 a 1,05 justamente por causa desse efeito.

9.6 A decisão sobre escala física — 2,5 minutos

O que mostrar

- Os dois CSVs de demonstração lado a lado: `outputs/predictions/demo_referencia/` e `outputs/predictions/demo_referencia_cm/`. **Os valores em pixels são idênticos; só o segundo tem as colunas em centímetros preenchidas.**
- A classe `PixelScale` em `measure.py`.
- A linha `DEFAULT_PIXELS_POR_CM = None` no final de `config.py`.

> **O QUE FALAR**
>
> *Essa é a decisão de que eu mais me orgulho, e é contraintuitiva, então deixa eu explicar o raciocínio.*
>
> *O resultado clinicamente útil é em centímetros. Ninguém quer saber que a ferida tem vinte e dois mil pixels quadrados. Mas o dataset que eu usei não tem nenhuma informação de escala: não tem EXIF confiável, não tem marcador de referência nas fotos, não tem registro da distância da câmera até a ferida.*
>
> *Eu poderia ter estimado um fator plausível e entregue tudo em centímetros. Ficaria mais bonito na demonstração. Eu decidi não fazer isso. Num contexto clínico, um número inventado é pior que nenhum número, porque ele parece confiável e vai ser usado para decidir tratamento.*
>
> *Então, por padrão, tudo sai em pixels e pixels quadrados. O `DEFAULT_PIXELS_POR_CM` é `None`, todos os campos em centímetros ficam nulos, e a saída em texto avisa explicitamente que não há conversão física disponível.*
>
> *Quando o usuário informa a escala, aí sim a conversão acontece, através de uma classe dedicada, o `PixelScale`, que é imutável e valida a entrada — escala zero, negativa, NaN ou infinita levantam erro na hora, em vez de gerar números sem sentido lá na frente. Ela tem quatro construtores: fator direto em pixels por centímetro, em pixels por milímetro, a partir de DPI para imagens de escâner, e — o método que eu recomendo em campo — a partir de um objeto de referência: você fotografa a ferida com uma régua ou um adesivo de tamanho conhecido, no mesmo plano da ferida, mede o objeto em pixels na imagem e informa o tamanho real.*
>
> *Esses dois relatórios aqui são exatamente as mesmas máscaras e as mesmas medidas em pixels. A única diferença é que no segundo eu informei trinta e oito pixels por centímetro. Repare que as colunas em pixels são idênticas e só as colunas em centímetro foram preenchidas — a conversão é uma camada por cima, ela não reprocessa nem altera a medição.*

Pontos técnicos importantes

- A unidade canônica interna é **pixels por centímetro**; milímetros e polegadas são derivados — adicionar uma unidade nova não exige mexer na medição.
- `PixelScale.resolve()` aceita objeto, número ou `None`, o que permitiu adicionar o sistema sem quebrar a assinatura antiga.

- Um teste verifica **campo a campo** que nenhum valor físico é fabricado sem escala.

Possíveis perguntas neste ponto

Você não poderia estimar a escala pelo tamanho típico de um membro, ou por aprendizado?

Poderia tentar, mas eu precisaria validar essa estimativa contra medidas reais, e eu não tenho esse dado. Sem validação, seria um chute com aparência de medida. A saída honesta foi entregar pixels por padrão e oferecer quatro formas fáceis de informar a escala real. Uma alternativa que está nas melhorias futuras é detectar automaticamente um marcador de referência na foto — um adesivo circular ou um ArUco — o que resolveria isso sem chute.

E o erro de perspectiva? Se a foto for tirada de lado?

Não é corrigido, e isso está declarado nas limitações conhecidas. O fator de escala vale para o plano do marcador de referência; foto oblíqua encurta a ferida na direção da inclinação, e superfície curva, como calcanhar, é medida como projeção plana, o que subestima a área real. A correção adequada seria homografia a partir de um marcador com quatro pontos conhecidos — está na lista de melhorias.

9.7 Qualidade e testes — 2,5 minutos

O que mostrar

- `tests/conftest.py` — as máscaras sintéticas.
- `tests/test_measure.py` — mostrar `test_rotation_invariance` e `test_without_scale_physical_fields_are_none`.
- `tests/test_pipeline_integration.py` — a classe `_StubSegmenter`.
- Se conseguir rodar: a saída do `pytest`.

> **O QUE FALAR**
>
> *O projeto tem cinquenta e quatro testes automatizados. Quarenta e cinco de medição e nove de integração ponta a ponta. Antes desse trabalho não havia nenhum teste.*
>
> *O desafio aqui era testar um sistema de visão computacional sem depender de um modelo treinado, de GPU e do dataset baixado. Eu resolvi de três formas.*
>
> *Primeira: as máscaras de teste são sintéticas, com geometria conhecida analiticamente. Eu não comparo com um valor que o código produziu antes — eu sei que o retângulo de cento e vinte por sessenta tem exatamente sete mil e duzentos pixels de área, e o teste verifica esse número. Isso é teste de correção, não de regressão.*
>
> *Segunda: para os testes de integração, eu escrevi um segmentador determinístico, o `_StubSegmenter`. É um módulo do PyTorch que devolve logits altos dentro de um retângulo conhecido e baixos fora dele. Com ele eu consigo testar o fluxo completo — leitura da imagem, transformação, inferência, reescala, limpeza, medição, geração dos artefatos — sem nenhum checkpoint treinado e sabendo qual é a resposta correta.*
>
> *Terceira: os testes de medição rodam sem PyTorch. Foi para isso que eu movi o import do torch para dentro das funções no `utils.py`. Bastam OpenCV e NumPy.*
>
> *Sobre o que os testes cobrem: uma ferida, múltiplas feridas, formato irregular em L, ruído e buracos, invariância à rotação, ausência total de ferida, máscara de baixa resolução, os formatos de máscara zero-um e zero-duzentos-e-cinquenta-e-cinco, booleana e float, rejeição de escalas inválidas, e compatibilidade com a API antiga. Esse último ponto é importante: eu tenho testes que fixam o comportamento da função antiga, para garantir que a expansão não quebrou o que já funcionava.*

Pontos técnicos importantes

- Testes de **correção analítica**, não de regressão — o valor esperado é derivado da geometria, não capturado de uma execução anterior.
- `pytest.importorskip` faz os testes de integração serem pulados com elegância quando a pilha de deep learning não está instalada.
- Testes de **não-regressão de contrato** na API antiga são a prova de que a compatibilidade foi tratada com intenção.

Possíveis perguntas neste ponto

Qual é a cobertura de código?

Eu não tenho o número medido — não há relatório de cobertura no projeto, e seria desonesto estimar. O que eu posso afirmar é a cobertura funcional: os cinquenta e quatro testes exercitam todas as funções públicas do módulo de medição, os três pontos de entrada da inferência e os cenários de borda. Rodar `pytest-cov` e fixar um patamar mínimo está na minha lista de melhorias.

Você tem teste com imagens reais de feridas?

Não com verificação de valor, e por um motivo: numa foto real eu não tenho a área verdadeira em pixels para comparar — teria que confiar na própria máscara, o que tornaria o teste circular. Por isso os testes de exatidão usam geometria sintética conhecida, e a validação em imagem real acontece pelo Dice e IoU no conjunto de teste, em `evaluate.py`.

9.8 Limitações — 1,5 minuto

O que mostrar

- A seção “Limitações conhecidas” do `README.md`.

> **O QUE FALAR**
>
> *Eu documentei as limitações no README, e acho que isso é parte do trabalho — um sistema de medição sem declaração de limites não é confiável.*
>
> *As principais são: sem escala física por padrão, como eu já expliquei; perspectiva e distância não são corrigidas; superfície curva é medida como projeção plana e subestima a área; o perímetro é levemente superestimado porque o contorno percorre a borda em escada; feridas satélite muito próximas podem ser fundidas em um componente só; e baixa resolução aumenta o erro relativo, porque a medição é confiável só até a granularidade do pixel.*
>
> *Tem uma que eu quero mencionar de forma bem direta, porque apareceu numa execução real e está registrada nos logs: com o limiar de ruído padrão, uma imagem de teste produziu nove componentes, e vários deles eram filetes de um a três pixels de largura, colados na borda da imagem — claramente artefato do modelo, não ferida. Eles passaram no filtro porque a área deles ficou logo acima do limiar. Isso mostra que o limiar padrão é permissivo para saída de modelo real, e que um critério adicional de forma — descartar componentes muito alongados e finos, ou encostados na borda — melhoraria o resultado. É uma correção que eu identifiquei mas não implementei.*

> **Por que este bloco vale a pena**
>
> Assumir uma limitação concreta, com evidência nos seus próprios logs, e já apresentar o diagnóstico da causa e a correção adequada é o sinal mais forte de domínio técnico que você pode dar. Vale mais que uma funcionalidade a mais.

9.9 Encerramento — 1 minuto

O QUE FALAR

Resumindo o que foi feito: o projeto partiu de uma pilha de segmentação já funcional — download do dataset, augmentations, U-Net, treino com early stopping e avaliação — e de uma medição rudimentar, que reportava só a maior ferida da imagem, sem tratamento de ruído e sem unidades.

O que eu construí em cima disso foi: medição de múltiplas feridas por imagem, sete novas métricas morfológicas, um sistema de escala física validado com quatro formas de informar o fator, um pós-processamento configurável que remove ruído sem deslocar a borda, anotação visual das medidas sobre a imagem, uma camada de relatório em CSV, e uma suíte de cinquenta e quatro testes onde antes não havia nenhum. Tudo isso mantendo a API antiga funcionando, com testes que fixam esse contrato.

E a decisão que eu levo como a mais importante do trabalho não foi acrescentar uma funcionalidade, foi recusar uma: o sistema não inventa centímetros. Quando não há escala, ele diz que não há.

10. Roteiro de demonstração prática

Estado real do projeto nesta máquina — verificado

Situação **após a preparação descrita na seção 14**:

• **Dataset presente e verificado:** 2.760 pares em `dataset/images/` e `dataset/masks/`, mais 664,8 MB em `dataset/raw/`.
• **Pilha completa instalada:** torch 2.13.0+cpu, smp, torchmetrics, albumentations, OpenCV, NumPy, pandas, scikit-learn, pytest.
• **Todos os testes passando:** `pytest tests/ -q` devolve **54 passed** — inclusive os 9 de integração, que antes eram pulados.
• **Artefatos reais gerados** em `outputs/predictions/dataset_ground_truth/` e `outputs/reports/`, a partir das máscaras verdadeiras do dataset.
• **Diagnóstico disponível:** `python verificar_ambiente.py` relata o estado da máquina e o que falta.

O que continua indisponível: não há checkpoint treinado (`checkpoints/` só tem `.gitkeep`), então `predict.py`, `evaluate.py` e `batch_inference.py` não têm modelo para carregar. A GPU é uma **GTX 680 (Kepler, sm_30)**, fora do suporte do PyTorch 2.x — o treino cairia para 4 núcleos de CPU e levaria dias. **Treinar aqui não é viável.**

10.1 Plano A — demonstração ao vivo (exige preparação antecipada)

Reserve tempo para isto **com antecedência**, nunca no dia. O download das dependências do PyTorch sozinho passa de 2 GB.

Preparação (fazer antes, com calma)

1. **Verifique a versão do Python.** O ambiente tem 3.14.6, e o `README` recomenda 3.11. Nem todas as versões do PyTorch publicam pacotes para versões muito recentes do Python — se a instalação falhar, instale o Python 3.11 e crie o ambiente com ele. Os artefatos `__pycache__/*.cpython-314.pyc` indicam que o projeto já rodou em 3.14 em outra máquina, então provavelmente funciona.
2. Crie o ambiente e instale: `python -m venv .venv`, ative-o, e rode `pip install -r requirements-dev.txt` (que já inclui os requisitos de execução).
3. **Obtenha um checkpoint.** Sem ele, `predict.py` não roda. São duas opções: recuperar o arquivo `.pt` da máquina onde o treino foi feito, ou treinar novamente — o que exige antes baixar o dataset e configurar as credenciais do Kaggle.
4. **Faça um ensaio completo** dos comandos abaixo e guarde as saídas.

Sequência da demonstração ao vivo

#	Ação	Comando / arquivo	O que dizer enquanto roda
1	Mostrar a estrutura	`README.md` e a raiz do projeto	"Cada módulo tem uma responsabilidade só; a configuração é toda centralizada."
2	**Rodar os testes**	`pytest tests/ -q`	"Cinquenta e quatro testes. Repare que os de medição não precisam de PyTorch nem de modelo treinado." **Comece por aqui: é rápido, é verde e não depende de checkpoint.**
3	Predição sem escala	`python predict.py --image <img> --checkpoint <ckpt>`	"Sem escala informada, tudo sai em pixels — e o sistema avisa isso explicitamente."
4	Mostrar a imagem anotada	`outputs/predictions/<nome>_medicoes.png`	"O amarelo é a bounding box alinhada aos eixos; o ciano é o retângulo de área mínima; vermelho e azul são os eixos de comprimento e largura."

#	Ação	Comando / arquivo	O que dizer enquanto roda
5	Mostrar o CSV	`outputs/predictions/<nome>_medicoes.csv`	“Uma linha por ferida, dezessete métricas. As colunas em centímetro estão vazias — isso é intencional.”
6	**Repetir com escala**	O mesmo comando + `--referencia-pixels 190 --referencia-cm 5`	“Mesmas máscaras, mesmas medidas em pixels. Só as colunas em centímetro foram preenchidas. A conversão é uma camada por cima da medição.”
7	Abrir `measure.py`	Função `clean_mask()`	Explicar a decisão de manter a morfologia desligada (bloco 9.5).
8	Inferência em lote	`python batch_inference.py --input-dir <pasta> ...`	“Processa a pasta inteira e consolida dois CSVs. Se uma imagem falhar, ela é registrada em log e o lote continua.”

10.2 Plano B — demonstração com o dataset real (recomendado)

Este plano **funciona hoje, do jeito que a máquina está**. Depois da preparação da seção 14, você tem o dataset completo, a suíte de testes rodando e medições reais sobre feridas reais. **A única coisa que falta é o modelo treinado** — e, como a medição opera sobre qualquer máscara binária, você demonstra o pipeline inteiro usando as máscaras verdadeiras do dataset no lugar das previstas. Bem conduzido, é mais convincente que o Plano A, porque tudo roda ao vivo.

#	Comando ao vivo	O que isso prova
0	`pytest tests/ -q`	**Abra com isto.** 45 passed, 1 skipped em ~1,5 s. Prova que o núcleo do projeto é testado e que a medição roda sem PyTorch — o *skip* do arquivo de integração é a evidência visível dessa decisão de arquitetura.
0b	Abrir `outputs/reports/exploracao_dataset.csv`	2.760 pares reais, 512×512. Ferida mediana = 1,55% da imagem: o argumento do desbalanceamento que justifica a perda Dice+BCE deixa de ser teórico.
0c	Abrir `outputs/reports/medicoes_ground_truth_por_ferida.csv`	3.414 feridas medidas em 2.760 imagens, em 23 segundos. Todas as colunas `*_cm` vazias — a política de não inventar escala, comprovada em escala real.

#	Abrir	O que demonstrar
1	`README.md`	Fluxo do pipeline, estrutura de pastas e a tabela das 17 métricas com as fórmulas.
2	`outputs/predictions/dataset_ground_truth/gt_maior_ferida_wsnet_1097_medicoes.png`	**O momento visual mais forte.** Ferida real de 127.568 px², com contorno verde, bounding box amarela, retângulo mínimo ciano e os eixos de comprimento (vermelho) e largura (azul). A diferença entre a caixa amarela e a ciano **prova visualmente** por que comprimento e largura vêm do `minAreaRect`.
2b	`gt_multiferida_medetec_0153_sem_rotulos.png`	Oito feridas medidas individualmente na mesma foto. Mostre também a versão *com* rótulos e assuma o defeito: os blocos de texto se sobrepõem quando há muitas feridas (ver seção 13, item 23).
3	`outputs/predictions/wound_000_comparacao.png`	O grid com os cinco painéis: original, máscara prevista, overlay e medições.

#	Abrir	O que demonstrar
4	`outputs/predictions/wound_000_medicoes.csv`	As 17 colunas de métrica, uma linha por ferida. Mostrar que as colunas `*_cm` estão vazias porque nenhuma escala foi informada.
5	**Os dois CSVs de demonstração**, lado a lado: `demo_referencia/` e `demo_referencia_cm/`	**O ponto alto da demonstração.** Valores em pixels idênticos nos dois; apenas o segundo tem as colunas em centímetro preenchidas, com o fator de 38 px/cm. Prova visual de que a conversão é uma camada separada da medição.
6	`logs/predict.log`	Registro de uma execução real: um checkpoint com `val_dice=0.7029` e a saída de medição de nove componentes. Serve também para introduzir o bloco de limitações (9.8).
7	`measure.py` — `clean_mask()` e `_rotated_dimensions()`	As duas decisões técnicas centrais: morfologia desligada e `minAreaRect`.
8	`config.py` — `MeasurementConfig`	Mostrar o comentário que **justifica por escrito** a decisão sobre morfologia. Código que explica o porquê, não só o quê.
9	`tests/conftest.py` e `tests/test_measure.py`	As máscaras sintéticas de geometria conhecida e o teste de invariância à rotação.
10	`tests/test_pipeline_integration.py`	O `_StubSegmenter`: como testar o pipeline inteiro sem modelo treinado.

> **Um cuidado ao mostrar os artefatos**
>
> Os arquivos em `demo_referencia/` apresentam máscaras mais limpas (uma ou duas regiões por imagem) do que o `wound_000_medicoes.csv` da raiz, que tem nove regiões. **Não é possível determinar, apenas pelos arquivos, qual segmentador gerou cada conjunto.** Se perguntarem, responda exatamente isso — e aproveite para explicar que a diferença ilustra bem o efeito do ruído do modelo sobre a contagem de regiões. Não afirme que foram gerados pelo mesmo modelo.

10.3 Comandos de referência

Objetivo	Comando
Instalar (execução)	`pip install -r requirements.txt`
Instalar (com testes)	`pip install -r requirements-dev.txt`
Rodar os testes	`pytest tests/ -q`
Baixar e explorar dados	`python dataset.py` e depois `python explore_data.py`
Treinar	`python train.py`
Avaliar	`python evaluate.py --checkpoint checkpoints/best_<run>.pt`
Prever (em pixels)	`python predict.py --image <img> --checkpoint <ckpt>`
Prever (com escala direta)	`python predict.py --image <img> --checkpoint <ckpt> --pixels-por-cm 38.0`
Prever (com referência)	`python predict.py --image <img> --checkpoint <ckpt> --referencia-pixels 190 --referencia-cm 5.0`
Lote	`python batch_inference.py --input-dir <pasta> --checkpoint <ckpt> --output-dir <saida>`
TensorBoard	`tensorboard --logdir logs/tensorboard`

11. Perguntas que podem ser feitas

Trinta perguntas organizadas por tema, com respostas que você pode adaptar. Onde a resposta correta é “eu não sei” ou “não está medido”, o texto diz isso — e explica como você descobriria. Admitir um limite com um plano é sempre mais forte que improvisar um número.

11.1 Arquitetura e decisões de projeto

Por que esta arquitetura de scripts e não uma aplicação web?

Porque o objetivo é um pipeline de visão computacional reprodutível, e a organização em scripts com configuração centralizada mantém isso executável em qualquer lugar, inclusive no Colab, sem infraestrutura. A separação por camadas já deixa o caminho aberto: `predict_single_image()` é uma função pura que recebe caminhos e devolve um dicionário — expor isso como endpoint seria um arquivo novo, sem tocar no resto.

Como você garante que a configuração não fica espalhada pelo código?

Tudo vive em `config.py`, em quatro dataclasses. Os módulos importam `config` e leem de lá. As funções aceitam uma configuração opcional como parâmetro e caem no padrão global quando não recebem — o que torna os testes capazes de injetar uma configuração própria sem alterar o estado global. É o que faz `test_max_wounds_limit_is_respected`.

Por que a medição não depende de PyTorch?

Foi uma decisão deliberada de testabilidade. A medição é geometria computacional pura — OpenCV e NumPy bastam. Para isso, o import do torch no `utils.py` foi movido para dentro das funções que precisam dele. O resultado é que os quarenta e cinco testes de medição rodam em segundos, sem GPU, sem baixar dois gigabytes de dependência.

O que acontece se eu quiser trocar a U-Net por outra arquitetura?

Edito uma string em `config.MODEL_CFG.architecture`. A fábrica em `models.py` suporta `unet`, `unetplusplus`, `deeplabv3plus` e `fpn`, e valida a entrada levantando `ValueError` com a lista de opções válidas. Nenhum outro arquivo muda.

11.2 Modelo, treinamento e algoritmos

Qual foi o desempenho do modelo?

O log de predição registra um checkpoint com `val_dice = 0.7029` — Dice de 0,70 no conjunto de validação. **Preciso ser transparente:** esse checkpoint não está na pasta do projeto, e o histórico de treinamento em CSV também não. Então esse é o único número de desempenho que eu tenho evidência para citar, e ele vem do log, não de um relatório de avaliação. As métricas completas no conjunto de teste sairiam de `evaluate.py`, que gera IoU, Precision, Recall, F1, Accuracy, matriz de confusão e curvas ROC e Precision-Recall.

Por que Dice + BCE e não só uma das duas?

A BCE otimiza pixel a pixel e é numericamente estável, mas é dominada pela classe majoritária — e nas imagens de ferida o fundo é a esmagadora maioria dos pixels, então um modelo que prevê “tudo fundo” já teria BCE baixa. A Dice mede a sobreposição da região e é insensível a esse desbalanceamento, mas é mais instável no início do treino, quando a interseção é próxima de zero. Combinadas meio a meio, uma compensa a fraqueza da outra.

Por que o Dice de validação é o critério de seleção e não a perda?

Porque a perda é uma combinação ponderada de dois objetivos e não é diretamente interpretável — uma perda menor não significa necessariamente uma segmentação melhor para o meu propósito. O Dice mede exatamente o que interessa: quanto a máscara prevista coincide com a real.

Como o overfitting é evitado?

Por quatro mecanismos: augmentation agressivo (flips, rotação até 35°, ShiftScaleRotate, CLAHE, brilho/contraste, blur, variação de matiz), encoder pré-treinado com taxa de aprendizado baixa, weight decay, e early stopping monitorando o Dice de validação com paciência 10. Além disso, o conjunto de teste é separado com seed fixa e nunca é usado durante o treino.

Por que a sigmoide não está dentro do modelo?

Por estabilidade numérica. O `BCEWithLogitsLoss` combina sigmoide e log em uma única operação, usando o truque do log-sum-exp, que evita overflow. Calcular sigmoide e depois log separadamente pode saturar. Por isso `activation=None` na configuração do modelo, e a sigmoide é aplicada explicitamente na inferência.

O que exatamente faz o mixed precision?

Executa parte das operações em ponto flutuante de 16 bits em vez de 32, o que reduz o uso de memória e acelera o treino em GPUs com Tensor Cores. O `GradScaler` escala a perda para evitar que gradientes pequenos virem zero em 16 bits. No projeto, ele é ativado apenas quando há GPU — em CPU é desligado automaticamente.

Por que gradient clipping?

Para evitar que um lote atípico produza um gradiente muito grande e desestabilize os pesos. Sob AMP, o código faz `scaler.unscale_()` antes de cortar — se cortasse o gradiente ainda escalado, o limiar de 1,0 não teria o significado pretendido. É um detalhe fácil de errar.

11.3 Medição e algoritmos geométricos

Como a área é calculada — por contagem de pixels ou pelo contorno?

Por **contagem de pixels** do componente, que é exato. Não uso `cv2.contourArea` para a área da ferida, porque o polígono do contorno aproxima a região e daria um valor ligeiramente diferente. O `contourArea` é usado só para a área do fecho convexo, no cálculo da solidez.

Por que a solidez às vezes passa de 1?

Porque a área é contagem de pixels e a área convexa é a área do polígono do fecho convexo — duas formas diferentes de medir área, e em componentes muito pequenos ou muito finos elas divergem, podendo dar razão acima de 1. Está documentado nas limitações. Foi mantido por compatibilidade com o comportamento original; a correção seria calcular a área convexa também por contagem de pixels, preenchendo o fecho numa máscara.

Como você mede o comprimento de uma ferida com formato irregular?

Pelo maior lado do retângulo de área mínima, que é invariante à rotação e reproduz a convenção clínica de maior comprimento por maior largura perpendicular. Reporto também o diâmetro de Feret máximo, que é a maior distância entre dois pontos da borda — para formas muito côncavas, como um “L”, os dois valores divergem, e essa divergência em si é informativa.

Qual a diferença entre circularidade, solidez e extensão?

Circularidade compara a forma com um círculo perfeito, via $4\pi \cdot \text{área}/\text{perímetro}^2$ — é sensível à irregularidade da borda. Solidez é a área dividida pela área do fecho convexo, e mede concavidade: um “L” tem solidez baixa, um círculo tem solidez próxima de 1. Extensão é a área dividida pela área da bounding box, e mede o quanto a forma preenche seu retângulo envolvente. As três capturam aspectos diferentes do formato.

Por que conectividade 8 e não 4?

Conectividade 8 considera vizinhos diagonais, então uma ferida com uma constrição diagonal fina de um pixel continua sendo um único componente. Com conectividade 4, ela seria dividida em duas feridas. Como a fragmentação artificial é o erro mais grave nesse contexto, 8 é a escolha conservadora. O parâmetro está exposto em `MeasurementConfig.connectivity`.

O preenchimento de buracos não pode inflar a área artificialmente?

Pode, se o buraco for real — por exemplo, uma ilha de tecido íntegro dentro da ferida. É uma escolha de compromisso: na segmentação de feridas, é muito mais comum o modelo classificar erroneamente tecido de granulação ou fibrina como fundo do que existir tecido íntegro completamente cercado por ferida. Por isso o preenchimento é o padrão — e é desligável em `MeasurementConfig.fill_holes`.

11.4 Dados, banco de dados e APIs

Qual banco de dados o projeto usa?

Nenhum. A persistência é o sistema de arquivos: as imagens e máscaras ficam em pastas, os resultados em CSV, e os pesos do modelo em arquivos `.pt`. Para o padrão de uso atual — lote e análise posterior — é suficiente. Um banco faria sentido se o sistema passasse a acompanhar pacientes ao longo do tempo, o que é uma das melhorias que eu listo.

Existe alguma API sendo consumida?

Uma: a API do Kaggle, usada apenas para baixar o dataset. O contrato efetivo é `KaggleApi().authenticate()` seguido de `dataset_download_files()` com `unzip=False` — o projeto descompacta manualmente e apaga os `.zip`. As credenciais vêm de `~/.kaggle/kaggle.json` ou das variáveis de ambiente, e nunca ficam no repositório. Se o pacote não estiver instalado, o `try/except ImportError` converte a falha numa mensagem que diz o que fazer.

Como imagens e máscaras são pareadas?

Por nome-base, em `list_image_mask_pairs()`, removendo os sufixos `_mask` e `-mask`. Se o pareamento por nome falhar completamente, há um *fallback* posicional que assume mesma contagem e mesma ordem alfabética — e registra um aviso em log. **Sendo honesto: esse fallback é frágil**, porque um pareamento errado ali contaminaria silenciosamente todo o treino. Eu o listo como ponto de atenção; o correto seria falhar em vez de adivinhar.

Como a divisão treino/validação/teste é garantida estável?

Um `random.Random(seed)` local embaralha a lista antes de fatiar, com a seed fixa em `config.SEED = 42`. Como a lista de pares vem ordenada de forma determinística, a mesma imagem cai sempre na mesma partição — inclusive entre execuções diferentes, o que é essencial para que o conjunto de teste não vaze para o treino ao longo de vários experimentos.

11.5 Segurança

Que preocupações de segurança este projeto tem?

Duas relevantes. A primeira é credenciais: as chaves do Kaggle nunca estão no código — vêm de variável de ambiente ou do arquivo padrão do Kaggle, fora do projeto. A segunda é dado sensível: fotografias clínicas de feridas são dado de saúde e, no Brasil, dado pessoal sensível pela LGPD. **O projeto hoje não implementa nenhum controle sobre isso** — não há anonimização, criptografia em repouso, controle de acesso nem trilha de auditoria. Para uso real com pacientes, isso seria obrigatório, e está na minha lista de melhorias.

Há risco em carregar um checkpoint `.pt`?

Sim, e é um ponto legítimo. `torch.load` usa `pickle`, que pode executar código arbitrário ao desserializar um arquivo malicioso. No projeto, o checkpoint é gerado pelo próprio treino, então o risco é baixo. Mas se o sistema passasse a aceitar checkpoints de terceiros, o correto seria usar `weights_only=True` no carregamento. É uma melhoria de uma linha que eu identifiquei.

O sistema valida as entradas?

Em parte. `predict_mask` levanta `FileNotFoundError` se a imagem não puder ser lida; `run_batch_inference` valida a existência da pasta e a presença de imagens; `PixelScale` rejeita escalas inválidas; e `build_model` valida arquitetura e encoder. O que não há é validação de tamanho ou de conteúdo do arquivo de imagem — em um serviço exposto à internet, isso seria necessário.

11.6 Performance

Quanto tempo leva para processar uma imagem?

Para a **etapa de medição eu tenho o número medido**: 2.760 máscaras de 512×512 foram lidas e medidas em **23,1 segundos — 119 imagens por segundo** em 4 núcleos de CPU, produzindo 3.414 medições de ferida. Ou seja, a medição é praticamente gratuita. O tempo da **inferência da U-Net eu não tenho medido**, porque o PyTorch não está instalado nesta máquina; é ela que domina o custo total e é muito mais rápida em GPU. Então a resposta completa é: a medição custa ~8 ms por imagem, e o total depende da inferência, que eu ainda não cronometrei.

O lote processa em paralelo?

Não. `run_batch_inference` processa uma imagem por vez, em laço sequencial, com uma inferência por chamada. A otimização evidente é agrupar as imagens em lotes reais de GPU em vez de uma por vez — está nas melhorias futuras. O treino, esse sim, usa `DataLoader` com `num_workers` para carregar em paralelo.

Onde estão os gargalos?

Três, na minha leitura. Primeiro, a inferência imagem a imagem no lote, sem agrupamento. Segundo, o `evaluate.py` acumula todas as probabilidades em memória para as curvas — e por isso subamostra com passo 37; a solução melhor seria acumular um histograma em vez dos valores brutos. Terceiro, a organização do dataset copia arquivos com `shutil.copy2`, duplicando o dataset em disco; links simbólicos evitariam isso.

Por que a subamostragem com passo 37 nas curvas?

Por memória: o conjunto de teste inteiro a 512×512 gera dezenas de milhões de pixels, e `roc_curve` ordena o vetor todo. O 37 é primo, o que evita alinhar com a largura da imagem e amostrar sempre a mesma coluna. E isso afeta **só as curvas** — as métricas agregadas são calculadas pelo TorchMetrics sobre todos os pixels.

11.7 Qualidade, processo e problemas encontrados

Onde está o repositório Git?

O projeto não está versionado em Git. Não existe pasta `.git`, portanto não há histórico de commits. Eu reconheço que isso é uma falha de processo — sem histórico, não há como auditar a evolução do código nem reverter uma mudança. Inicializar o repositório, com `.gitignore` excluindo `dataset/`, `checkpoints/`, `outputs/` e `logs/`, é a primeira coisa que eu faria.

Qual foi o bug mais difícil que você resolveu?

Não foi um bug de *crash*, foi um de consistência silenciosa: a imagem anotada podia discordar do CSV. A medição limpa a máscara internamente por padrão, mas a função de anotação recebe a máscara separadamente — então era possível desenhar o contorno de uma máscara e reportar os números de outra. Ninguém veria um erro; simplesmente os números não bateriam com o desenho. A solução foi o parâmetro `cleanup`: os scripts limpam uma vez e passam a mesma máscara para as duas coisas. Escrevi dois testes para fixar isso — um de equivalência e um de idempotência.

Você encontrou algum problema que não corrigiu?

Sim, e prefiro citá-los. O limiar de ruído padrão é permissivo para saída de modelo real — numa execução, filetes de um a três pixels colados na borda passaram como feridas; faltaria um critério de forma. O `evaluate.py` mede sobre a máscara de 512×512, e não na resolução original como o `predict.py`, então as medidas dos dois não são diretamente comparáveis. E o valor de retorno do `predict_single_image` mistura a ferida principal medida na máscara crua com a lista de feridas medida na máscara limpa. Todos estão listados na seção de pontos de atenção do documento.

Como você garantiu que não quebrou o que já existia?

Escrevendo testes que fixam o comportamento antigo antes de expandir. O `test_legacy_measure_wound_ignores_cleanup` verifica que a função antiga continua **não** limpando a máscara — o buraco de 36 pixels continua descontado da área. O `test_empty_mask_legacy_raises` garante que ela continua levantando `ValueError`. E o teste do lote verifica explicitamente que os arquivos antigos ainda são gerados. A compatibilidade não foi uma esperança, foi uma asserção.

Se você tivesse mais uma semana, o que faria?

Três coisas, nessa ordem. Primeiro, inicializar o Git — é a lacuna de processo mais grave. Segundo, o filtro de forma para eliminar os artefatos alongados que hoje passam como feridas, porque é um problema de qualidade do resultado que eu já diagnostiquei. Terceiro, um estudo de concordância comparando as medidas do sistema com medição manual, que é o que transformaria isso de projeto técnico em ferramenta com credibilidade clínica.

12. Melhorias futuras

> **Estas são sugestões, não funcionalidades existentes**
>
> **Nada nesta seção está implementado no projeto.** Apresente-as sempre no futuro do subjuntivo — "o próximo passo seria…". Confundir plano com entrega é o erro mais caro que se pode cometer numa apresentação técnica.

12.1 Prioridade alta — corrigem lacunas identificadas

Melhoria	Problema que resolve
Versionar o projeto em Git	Hoje não há histórico, autoria nem possibilidade de reverter. Com `.gitignore` para `dataset/`, `checkpoints/`, `outputs/` e `logs/`.
Filtro de forma no pós-processamento	Componentes muito alongados e finos, ou colados na borda da imagem, passam hoje como feridas. Um critério de razão comprimento/largura combinado com proximidade da borda eliminaria esses artefatos — o problema está evidenciado nos próprios logs.
Unificar a resolução de medição	`evaluate.py` mede sobre a máscara de 512×512 e `predict.py` sobre a resolução original, tornando os relatórios não comparáveis.
Padronizar a máscara do valor de retorno	Em `predict_single_image`, a ferida principal vem da máscara crua e a lista de feridas vem da limpa, podendo divergir.
Medir cobertura de testes	Adicionar `pytest-cov` e fixar um patamar mínimo; hoje a cobertura não é conhecida.
Atualizar APIs depreciadas — **FEITO**	AMP e `np.trapz` já corrigidos, e as versões travadas com limite superior. Ver seção 15.
Substituir `ShiftScaleRotate`	Único aviso de depreciação que resta: o albumentations 2.x pede `A.Affine` no lugar. Ainda funciona, mas pode sair na versão 3.

12.2 Prioridade média — ampliam a capacidade

Melhoria	Ganho esperado
Detecção automática de marcador de referência	Reconhecer um adesivo circular ou um marcador ArUco na foto eliminaria a entrada manual da escala — a maior fricção de uso do sistema hoje.
Correção de perspectiva por homografia	Um marcador com quatro pontos conhecidos permitiria retificar o plano e corrigir fotos oblíquas, atacando uma das limitações declaradas.
Separação de feridas encostadas	*Watershed* com marcadores, ou um modelo de segmentação de instâncias, resolveria a fusão de feridas satélite muito próximas.
Inferência em lotes reais de GPU	Agrupar as imagens em *batches* em vez de processar uma a uma aceleraria significativamente o processamento em lote.
Segmentação multiclasse de tecidos	Distinguir granulação, esfacelo e necrose daria informação clínica muito mais rica que a máscara binária atual. Exigiria um dataset anotado nesse nível.
Aumento de teste (TTA)	Média das predições sobre a imagem original e suas versões espelhadas costuma melhorar a estabilidade da borda — que é exatamente o que a medição consome.

12.3 Prioridade estratégica — o que faria disso um produto

- **Estudo de concordância clínica** — comparar as medidas do sistema com medição manual por profissionais, calculando Bland-Altman e coeficiente de correlação intraclasse. É o que transformaria o projeto de exercício técnico em ferramenta confiável, e nenhum outro item desta lista tem esse peso.
- **Acompanhamento temporal por paciente** — banco de dados relacional ligando imagens ao mesmo paciente, com gráfico de evolução da área ao longo do tempo. É esse gráfico, e não a medida isolada, que responde à pergunta clínica “o tratamento está funcionando?”.
- **Camada de conformidade com a LGPD** — anonimização, criptografia em repouso, controle de acesso e trilha de auditoria. Obrigatório antes de qualquer uso com dados reais de pacientes.
- **Interface de uso** — uma API REST com `FastAPI` ou uma interface simples para envio da foto, eliminando a linha de comando.
- **Estimativa de incerteza** — reportar uma faixa de confiança junto com a área, usando *Monte Carlo dropout* ou um *ensemble*. Uma medida clínica sem incerteza associada é menos útil do que parece.

13. Inconsistências e pontos de atenção

Registro técnico dos problemas, incoerências e decisões questionáveis encontrados na análise. **Nenhum deles impede a apresentação** — ao contrário: conhecê-los e assumi-los espontaneamente é uma vantagem. A ordem é decrescente por relevância.

13.1 Estado do ambiente

#	Constatação	Impacto
1	**O projeto não é um repositório Git**	Não há histórico, autoria nem diffs. Impossibilita a atribuição objetiva de mudanças e a reversão. Maior lacuna de processo do projeto.
2	**Nenhum checkpoint treinado** — `checkpoints/` só tem `.gitkeep`	`predict.py`, `evaluate.py` e `batch_inference.py` não podem ser executados. O log cita um modelo com `val_dice=0.7029` que não está mais presente. **Continua válido.**
3	Dataset ausente — **RESOLVIDO**	O dataset foi baixado e organizado: 2.760 pares verificados. Ver seção 14.
4	**PyTorch não instalado** (OpenCV/NumPy/pandas já estão)	A medição roda; a inferência e o treino, não. A GPU **GTX 680 (sm_30)** está fora do suporte do PyTorch 2.x, então treinar nesta máquina cairia para CPU e levaria dias — é inviável localmente.
5	Divergência de versão do Python	O `README` indica 3.11; os `.pyc` indicam execução em 3.14.
6	Nenhum histórico de treinamento salvo	`outputs/reports/` está vazio: não há `training_history_*.csv` nem figuras de avaliação para exibir.

13.2 Inconsistências no código

#	Problema	Onde	Consequência
7	Máscaras diferentes no mesmo retorno	`predict.py` (linhas 205 e 233), `batch_inference.py` (99 e 109),	A ferida principal do dicionário de retorno é medida na **máscara crua**, enquanto a lista `feridas` e o CSV vêm da **máscara limpa**. Os dois podem discordar para a mesma imagem.
8	Resolução de medição divergente	`evaluate.py` (linha 72)	Mede sobre a máscara de 512×512 do *dataloader*, enquanto `predict.py` mede na resolução original. As medidas em pixels dos dois relatórios **não são comparáveis**.
9	Limiar de ruído permissivo	`config.MeasurementConfig`	Em execução real registrada em log, nove componentes foram reportados, vários com 1 a 3 px de largura na borda da imagem. Falta um critério de forma além do de área.
10	Solidez pode exceder 1	`measure._measure_component`	Área por contagem de pixels dividida por área de polígono — grandezas medidas de formas diferentes. Valores de até 1,71 aparecem nos CSVs. Documentado, mas é uma incoerência real.

#	Problema	Onde	Consequência
11	Pareamento posicional como *fallback*	`dataset.list_image_mask_pairs`	Se o pareamento por nome falhar, o código pareia por ordem alfabética e apenas avisa em log. Um pareamento errado contaminaria todo o treino silenciosamente — seria preferível falhar.
12	Separação imagem/máscara por palavra-chave	`dataset._organize_folders`	Heurística sobre o caminho e o nome do arquivo. Uma imagem cujo nome contenha “gt” seria classificada como máscara.

13.3 Dependências e APIs depreciadas

#	Item	Onde	Observação
13	scikit-image declarada e não usada — **CORRIGIDO**	`requirements.txt`	Nenhum arquivo importava `skimage`. Removida da lista de dependências.
14	torchvision declarada sem import direto	`requirements.txt`	Provavelmente dependência transitiva de `smp`; declará-la diretamente é aceitável, mas não é usada pelo código do projeto.
15	API de AMP depreciada — **CORRIGIDO**	`train.py`	`train.py` passou a escolher entre `torch.amp` e `torch.cuda.amp` em tempo de execução, funcionando nas duas gerações do PyTorch. Verificado em torch 2.13: **zero avisos**.
16	`np.trapz` — **CORRIGIDO (era falha grave)**	`evaluate.py`	Não estava apenas depreciada: foi **removida no NumPy 2.0**, e `requirements.txt` pedia `numpy>=1.26` — ou seja, **toda instalação nova quebrava** ao gerar a curva ROC. Substituída por `sklearn.metrics.auc`, que faz a mesma integração trapezoidal e independe da versão do NumPy.
17	Import não utilizado	`metrics.py` (17),	`F1Score` é importado, mas apenas `BinaryF1Score` é usado.
18	Efeito colateral no import	`config.py` (37-42),	O módulo **cria sete diretórios** ao ser importado. Conveniente, mas significa que qualquer import de `config` escreve no disco — inclusive em testes.

13.4 Documentação e artefatos

#	Observação
19	A **etapa 14** da numeração do projeto não foi encontrada em nenhum arquivo. As demais (1 a 13 e 15) estão todas mapeadas.
20	Os artefatos em `outputs/predictions/demo_referencia/` apresentam máscaras substancialmente mais limpas (1 a 2 regiões) que o `wound_000_medicoes.csv` da raiz (9 regiões). **Não é possível determinar pelos arquivos qual segmentador gerou cada conjunto** — evite afirmar que vieram do mesmo modelo.
21	Os logs indicam que o projeto foi desenvolvido em outra máquina (os caminhos registrados apontam para outro usuário do Windows). Isso é irrelevante para o conteúdo técnico, mas explica por que checkpoint e dataset não acompanham a pasta.
22	Não há `.gitignore`, `pyproject.toml`, `setup.cfg`, `pytest.ini`, Dockerfile nem configuração de CI/CD. O projeto é executado inteiramente por scripts.

13.5 Problemas descobertos ao rodar sobre o dataset real

Os quatro itens abaixo **só apareceram quando o código foi executado sobre os dados verdadeiros** — nenhum deles é visível na leitura do código nem nos testes sintéticos. São os achados mais valiosos desta análise.

#	Problema	Evidência	Correção indicada
23	**Rótulos se sobrepõem** na imagem anotada	Em `medetec_0153.png`, com 8 feridas, os blocos de texto cobrem uns aos outros e a anotação fica ilegível. `_draw_label_block()` limita cada rótulo às bordas da imagem, mas **não trata colisão entre rótulos**.	Paliativo já existente: `draw_labels=False` desenha só a geometria e resolve o caso denso. A correção real seria um posicionamento que evite sobreposição, ou uma legenda lateral numerada.
24	**A divisão oficial do dataset é descartada**	O dataset traz `train_images/` (2.208) e `test_images/` (552) separados. `_organize_folders()` achata tudo em duas pastas e `split_dataset()` refaz um sorteio 70/15/15 sobre o conjunto inteiro — **imagens do teste oficial entram no treino**.	Preservar a partição de origem, ou ao menos documentar que a métrica obtida **não é comparável** com a de trabalhos que usam o split oficial.
25	**Circularidade e solidez acima de 1** em dados reais	Nas 3.414 feridas medidas: circularidade chega a **1,35** e solidez a **1,48**. Ambas deveriam ser limitadas a 1. Afeta sobretudo componentes pequenos.	Causa única: a área vem de **contagem de pixels** e o perímetro/fecho convexo vêm da **geometria do contorno**. Medir as duas grandezas da mesma forma eliminaria a incoerência.
26	**24 máscaras totalmente vazias** no dataset	São imagens sem nenhuma ferida anotada. `measure_wounds()` devolveu lista vazia nas 24, sem exceção, e as imagens continuaram presentes no relatório.	Nenhuma — **é a validação da decisão de contrato**. A API antiga `measure_wound()` levantaria `ValueError` em todas elas.

> **Como usar esta seção a seu favor**
>
> Escolha **dois ou três** itens desta lista e mencione-os espontaneamente no bloco 9.8 — de preferência os itens 1 (ausência de Git), 9 (limiar permissivo) e 7 ou 8 (as inconsistências de máscara e resolução). Apresentar um problema que você mesmo encontrou, com o diagnóstico da causa e a correção adequada, demonstra mais competência técnica do que qualquer funcionalidade adicional. E tira da banca a chance de descobri-lo primeiro.

14. Execução real com o dataset Kaggle

O dataset foi **baixado, organizado e efetivamente processado** pelo código do projeto. Esta seção registra o que foi executado e os números obtidos. Tudo aqui é resultado de execução real, não estimativa — e é o material mais forte que você tem para a apresentação, porque deixa de ser código que "deveria funcionar" e passa a ser resultado medido.

14.1 O que foi feito

#	Etapa	Resultado
1	Autenticação no Kaggle	Via `KAGGLE_API_TOKEN` (formato `KGAT_…`), suportado pelo cliente `kaggle 2.2.4`. **O token foi passado apenas por variável de ambiente** e não foi gravado em nenhum arquivo do projeto.
2	Download	664,8 MB em 63 s, extraídos para `dataset/raw/`.
3	Organização	5.520 arquivos classificados pela **mesma lógica** de `_organize_folders()`: 2.760 imagens e 2.760 máscaras. Foram usados **hardlinks** em vez de cópias — o disco tinha apenas 11 GB livres, e a cópia duplicaria 696 MB. É transparente para o código, que só lê os arquivos por caminho.
4	Verificação de integridade	**Zero colisões** de nome entre treino e teste; **zero** imagens sem máscara e zero máscaras sem imagem. O pareamento por nome de `list_image_mask_pairs()` casou os 2.760 pares — o *fallback* posicional (frágil) **não foi acionado**.
5	Suíte de testes	**45 passed, 1 skipped em 1,51 s**, com PyTorch ausente. Confirma na prática a decisão de arquitetura da seção 2.3.
6	Exploração	`outputs/reports/exploracao_dataset.csv` e o histograma `outputs/figures/distribuicao_area.png`.
7	**Medição real**	3.414 feridas medidas em 2.760 imagens, salvas em `outputs/reports/medicoes_ground_truth_por_ferida.csv`.
8	Imagens anotadas	Cinco casos representativos em `outputs/predictions/dataset_ground_truth/`.

> **Sobre a natureza destes resultados**
>
> As medições desta seção foram feitas sobre as **máscaras verdadeiras (ground truth) do dataset**, não sobre máscaras previstas por um modelo — porque não há checkpoint treinado disponível. **Diga isso explicitamente na apresentação.** Isso não enfraquece a demonstração: o módulo de medição opera sobre qualquer máscara binária, e usar a máscara verdadeira **isola a qualidade da medição da qualidade da segmentação** — o que na verdade é metodologicamente mais limpo para avaliar a etapa que você construiu.

14.2 Composição do dataset

Característica	Valor medido
Pares imagem/máscara	**2.760** (2.208 em `train_` + 552 em `test_` na origem)
Resolução	**512×512 para todas** — uniforme. O *resize* do pipeline é, portanto, um no-op neste dataset, e a reescala de volta em `predict.py` é a identidade.
Fontes	`fusc`: 1.210 · `wsnet`: 1.176 · `medetec`: 374
Área da ferida (px)	mediana **4.058** · média 10.087 · máx. 126.188 · mín. 0

Característica	Valor medido
% da imagem ocupado	mediana **1,55%** · média 3,85% · máx. 48,1%
Máscaras vazias	**24** imagens sem nenhuma ferida anotada
Feridas minúsculas	115 ocupam menos de 0,1% da imagem
Área média por fonte	`fusc`: 3.114 px (1,19%) · `wsnet`: 13.351 px (5,09%) · `medetec`: 22.381 px (8,54%)

O número que mais importa para a apresentação é a **mediana de 1,55%**. Ele transforma o argumento sobre a perda em fato: um modelo que previsse “tudo fundo” acertaria mais de 98% dos pixels. É exatamente contra isso que a Dice entra na perda combinada.

14.3 Resultado da medição

Métrica de execução	Valor
Imagens processadas	2.760
Tempo total	**23,1 s** — 119,4 imagens/s em 4 núcleos de CPU (~8 ms/imagem)
Feridas medidas	**3.414**
Imagens com mais de uma ferida	**445 (16,1%)**; máximo de **8** feridas em uma única imagem
Imagens sem ferida	24 — `measure_wounds()` devolveu lista vazia, **sem exceção**, e as imagens permaneceram no relatório
Colunas em cm preenchidas	**Nenhuma** — nenhuma escala foi informada, e o sistema não inventou nenhuma

Distribuição das medidas (3.414 feridas reais)

Métrica	Mínimo	Mediana	Média	Máximo
Área (px²)	20	2.720	8.342	127.568
Comprimento (px)	4,0	81,0	110,1	542,6
Largura (px)	2,8	49,0	67,5	398,8
Perímetro (px)	13,7	248,2	342,7	1.805,4
Circularidade	0,14	0,63	0,61	**1,35**
Solidez	0,45	0,96	0,94	**1,48**
Extensão	0,16	0,67	0,65	0,96

Os máximos em negrito são **anomalias reais**, não erros de leitura: circularidade e solidez deveriam ser limitadas a 1. Ambas têm a mesma causa — área por contagem de pixels dividida por grandezas derivadas da geometria do contorno. Ver seção 13.5, item 25. **Mencione isso antes que perguntem.**

14.4 O argumento mais forte da sua apresentação

O impacto medido da medição multi-ferida

Nas 2.760 imagens reais, foram detectadas **3.414 feridas**. A API antiga `measure_wound()`, que reporta apenas o maior componente de cada imagem, teria devolvido **2.736**.

Ou seja: **678 feridas — 19,9% de todas as feridas do dataset — seriam silenciosamente descartadas**, afetando **445 imagens (16,1%)**.

Este é o número que justifica sozinho a funcionalidade que você construiu. Não é um caso hipotético de borda: é um quinto de todas as feridas deste dataset. **Use exatamente esta frase na apresentação.**

14.5 Como reproduzir

Passo	Comando
Credencial	`export KAGGLE_API_TOKEN=KGAT_…` (nunca em arquivo versionado)
Instalar o mínimo	`pip install opencv-python-headless numpy pandas matplotlib pytest kaggle`
Baixar	`python dataset.py` — **requer também torch e albumentations**, porque `dataset.py` os importa no topo do arquivo
Validar	`pytest tests/ -q`

Uma limitação de arquitetura que este exercício expôs

Para baixar o dataset pelo caminho oficial (`python dataset.py`) é preciso ter **PyTorch e Albumentations instalados**, porque `dataset.py` os importa no topo do módulo — embora as funções de download e organização usem apenas `pathlib` e `shutil`. A mesma técnica de import adiado já aplicada em `utils.py` resolveria isso e permitiria baixar e organizar os dados sem a pilha de deep learning. **Isso foi implementado — ver seção 15.3.**

15. Portabilidade: rodar em qualquer computador

Esta seção documenta as correções feitas para que o projeto seja instalável e executável em qualquer máquina. **É material de apresentação de primeira ordem**: mostra que você não só construiu a funcionalidade, como se preocupou com o ciclo de vida do software. Todas as mudanças foram validadas com a suíte de testes.

15.1 A falha grave: o projeto estava quebrado em instalações novas

> **O bug mais importante encontrado em toda a análise**
>
> `evaluate.py` calculava a área sob a curva ROC com `np.trapz`. Essa função **não foi apenas depreciada: foi removida no NumPy 2.0**.
>
> Como o `requirements.txt` declarava `numpy>=1.26.0` sem limite superior, **qualquer instalação feita a partir de 2024 recebia NumPy 2.x e a avaliação quebrava** com `AttributeError` ao gerar a curva ROC. O projeto rodava na máquina original apenas porque lá havia um NumPy antigo.
>
> **Correção:** trocado por `sklearn.metrics.auc`, que faz a mesma integração trapezoidal, já era dependência do projeto e não depende da versão do NumPy. Verificado: curva ROC, curva PR e matriz de confusão geradas com NumPy 2.5.1.

Este é o exemplo perfeito de por que dependências sem limite superior são perigosas: o código não mudou, o ambiente mudou, e o projeto quebrou sozinho.

15.2 Versões travadas nos dois sentidos

Arquivo	Estratégia	Quando usar
`requirements.txt`	Faixas com **limite superior**: `>=x,<y`. Impede que uma nova major entre sozinha.	Instalação normal, em qualquer máquina.
`requirements-dev.txt`	O anterior mais `pytest`	Quando também for rodar os testes.
`requirements-verificado.txt`	Versões **exatas** (==) com que os 54 testes passam.	Quando quiser reproduzir exatamente o ambiente validado.

A dependência `scikit-image` foi **removida**: estava declarada mas nenhum arquivo do projeto a importava. `kaggle` subiu para `>=2.0`, que é o mínimo para aceitar o token moderno.

15.3 Baixar os dados sem a pilha de deep learning

`dataset.py` importava `albumentations` e `torch` no topo do módulo, embora o download, a organização das pastas, o pareamento e a divisão usem apenas `pathlib`, `shutil`, `cv2` e `pandas`. Na prática, era preciso instalar **1,5 GB de dependências para baixar um arquivo**.

Os imports passaram a ser tolerantes (`try/except ImportError`), expondo a flag `DEEP_LEARNING_DISPONIVEL`. As funções que realmente precisam da pilha falham com uma mensagem que diz o que instalar. Verificado em ambiente simulado sem torch: download, exploração, pareamento e divisão funcionam; `get_train_transforms()` e `WoundDataset` levantam `RuntimeError` explicativo. É a mesma técnica já usada em `utils.py` — agora aplicada com coerência nos dois módulos.

15.4 Compatibilidade de AMP entre gerações do PyTorch

O PyTorch 2.4 depreciou `torch.cuda.amp.autocast` e `torch.cuda.amp.GradScaler` em favor de `torch.amp`. `train.py` ganhou duas funções — `make_grad_scaler()` e `autocast_context()` — que detectam a API disponível **em tempo de execução**. O mesmo código roda tanto em instalações antigas quanto novas.

Verificado em torch 2.13: API nova detectada, **zero avisos de depreciação**.

15.5 Credenciais do Kaggle: três mecanismos

O README documentava apenas `kaggle.json` e o par `KAGGLE_USERNAME`/`KAGGLE_KEY`. O Kaggle introduziu desde então um **token moderno** (formato `KGAT_…`) lido de `KAGGLE_API_TOKEN`, que é o formato entregue hoje pelo site. Quem tentasse usar o token novo com as instruções antigas ficaria travado sem entender o motivo.

Os três mecanismos passaram a ser documentados no README e no docstring de `download_dataset()`, e a falha de autenticação agora produz uma mensagem que lista as três opções em vez de um *traceback* cru.

15.6 `verificar_ambiente.py` — o diagnóstico

Script novo que responde, em qualquer computador, à pergunta "o que falta aqui?". **Roda sem nenhuma dependência instalada** — usa apenas a biblioteca padrão — de modo que funciona em um Python recém-instalado, antes do primeiro `pip install`.

Verifica	Detalhe
Sistema	Versão do Python (exige 3.10+), sistema operacional, se há ambiente virtual ativo e espaço livre em disco.
Dependências	Cada pacote, com versão e aviso quando abaixo do mínimo, agrupados por finalidade (medição, deep learning, gráficos, infra, testes).
GPU	Detecta a placa e compara a *compute capability* com as arquiteturas que o PyTorch instalado realmente compilou. **É isso que revela o problema da GTX 680**: CUDA existe, mas a placa é sm_30 e o PyTorch só traz sm_50+.
Kaggle	Qual dos três mecanismos de credencial está configurado, se algum.
Dados	Contagem em `raw/`, `images/` e `masks/`, e quantos pares o pareamento por nome realmente forma.
Modelo	Checkpoints presentes em `checkpoints/`.
Veredito	Diz o que a máquina consegue fazer **agora** — medir, testar, baixar dados, treinar, prever — e lista os comandos seguintes na ordem certa. Sai com código 0 (pronto) ou 1 (falta algo), o que permite usá-lo em CI.

> **Como usar isso na apresentação**
>
> Rode `python verificar_ambiente.py` ao vivo, logo depois do `pytest`. São dez segundos e demonstram três coisas de uma vez: que o projeto é instalável em outra máquina, que você pensou na experiência de quem vai recebê-lo, e que você conhece as limitações do próprio hardware — a detecção da GPU incompatível é um detalhe que impressiona.

15.7 Resumo das mudanças

Arquivo	Mudança	Status
`evaluate.py`	`np.trapz` (removida no NumPy 2) trocada por `sklearn.metrics.auc`	**Corrige quebra**
`train.py`	AMP compatível com PyTorch antigo e novo em tempo de execução	Corrige depreciação
`dataset.py`	Imports tolerantes de torch/albumentations; três mecanismos de credencial documentados; erro de autenticação acionável	Melhoria

Arquivo	Mudança	Status
`requirements.txt`	Faixas com limite superior; `scikit-image` removida; `kaggle>=2.0`	Trava versões
`requirements-dev.txt`	Limite superior no pytest	Trava versões
`requirements-verificado.txt`	Novo: versões exatas validadas	**Novo**
`verificar_ambiente.py`	Novo: diagnóstico completo do ambiente	**Novo**
`README.md`	Instalação multiplataforma, variante CPU, política de versões, credenciais e o que não copiar ao mover o projeto	Documentação

Validação final: todos os módulos compilam, `pytest tests/ -q` devolve **54 passed**, e `pip install -r requirements-dev.txt` resolve sem conflito de dependências.